Computer Architecture
Pipelining

Tatpong Katanyukul

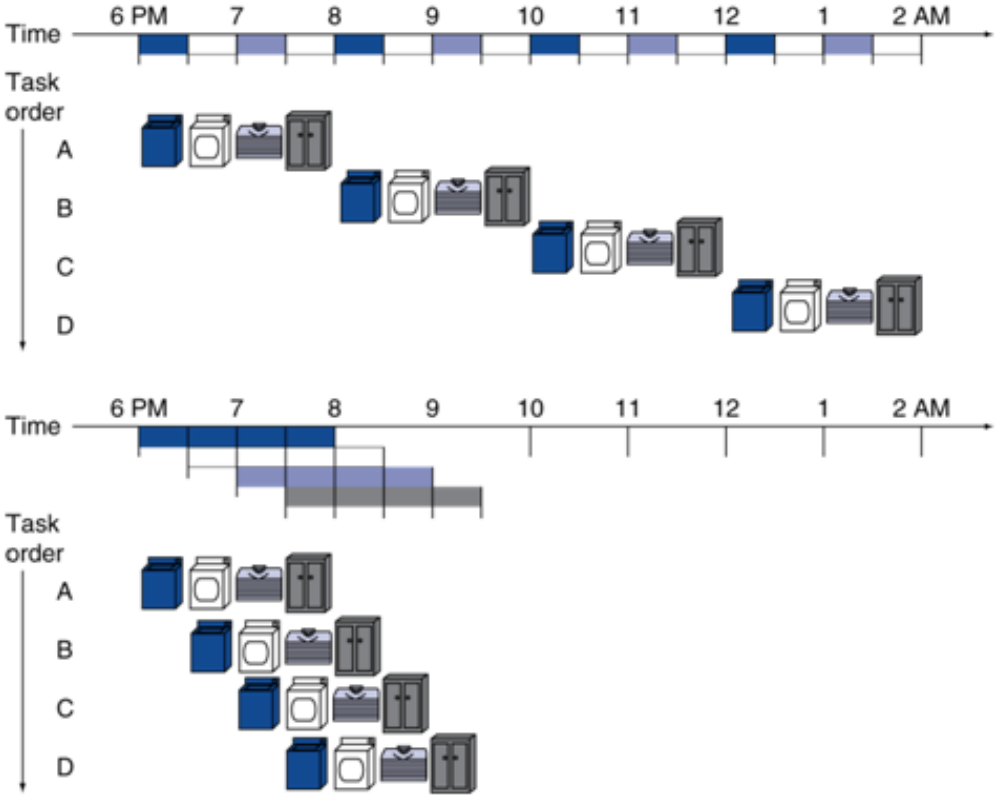
CoE, KKU

1 พฤษภาคม พ.ศ. 2558

Pipelining Analogy

- Pipelined laundry: overlapping execution

  Parallelism improves performance



- Four loads:
  - no pipelining: 8 hrs
  - pipelining: 3.5 hrs
  - speed up: $8/3.5 \approx 2.3$
- n loads:
  - speed up: $\frac{2n}{0.5n+1.5}$
  - for large n, speed up ≈ 4
  - speed up $\approx$ # stages

MIPS Pipeline

Five stages, one step per stage

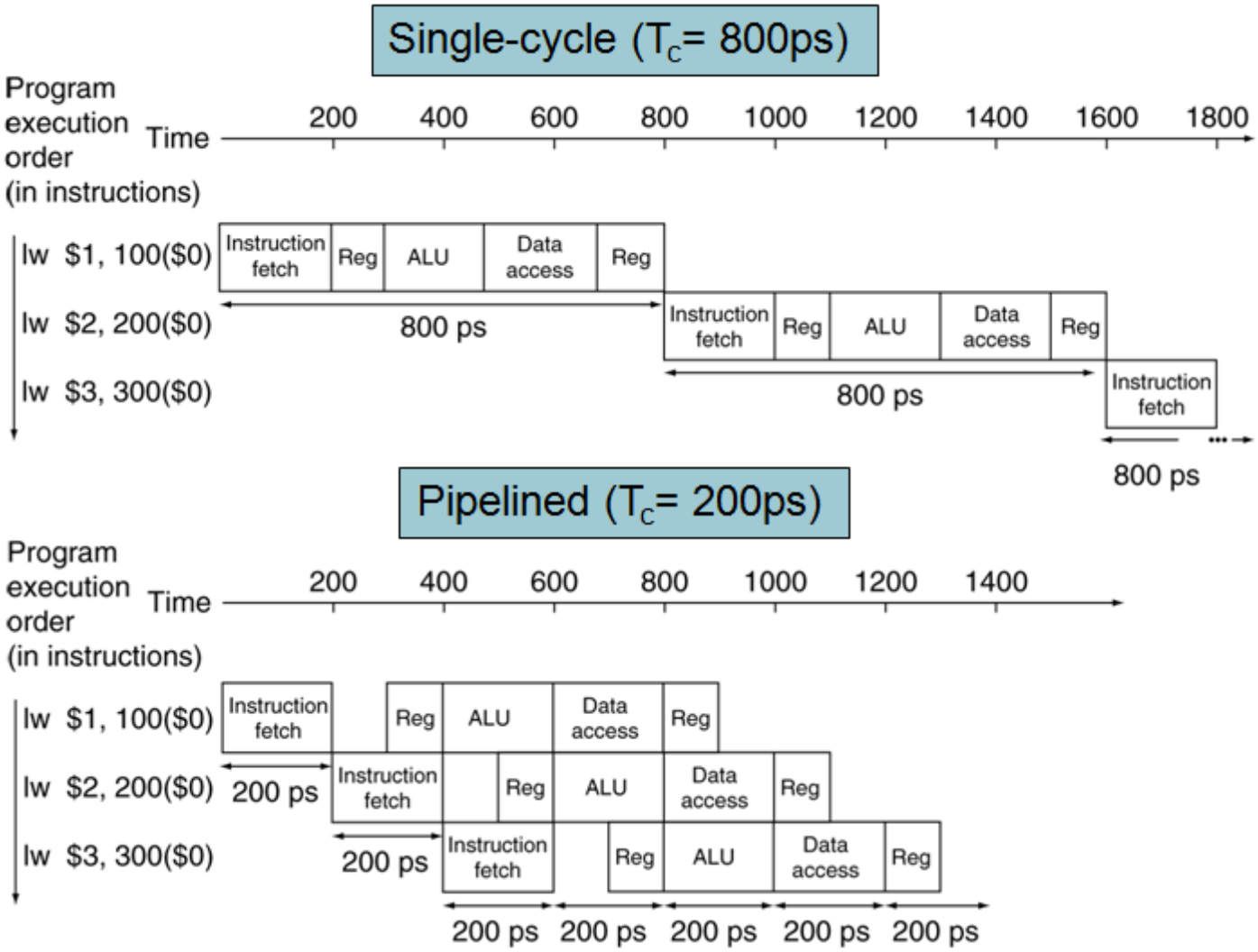
- IF: Instruction fetch from memory
- ID: Instruction decode & register read
- EX: Execute operation or calculate address
- MEM: Access memory operand
- WB: Write result back to register

Pipeline Performance

- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100ps	200ps	200ps	100ps	800ps
sw	200ps	100ps	200ps	200ps		700ps
R-type	200ps	100ps	200ps		100ps	600ps
beq	200ps	100ps	200ps			500ps

Pipeline Performance



Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time
  - Time between instruction, T_c

$$T_{c(pipelined)} = \frac{T_{c(nonpipelined)}}{\#stages}$$

- If not balanced, speedup is less
  - Need to allocate 200ps for register operation that takes only 100ps

$$T_{c(pipelined)} = \frac{800ps + 100ps + 100ps}{5} = 200ps$$

- Speedup due to increased throughput
- !!! Latency (time for each instruction) does not decrease

Pipelining and ISA Design

MIPS ISA designed for pipelining

- All instructions are 32-bits
  - Easier to fetch and decode in one cycle
- Few and regular instruction formats
  - Can decode and read registers in one step
- Load/store addressing
  - Can calculate address in 3rd stage, access memory in 4th stage

Hazards

Pipelining sounds sweet,

but what could make it NOT working?

Hazards

Pipelining sounds sweet,
but what could make it NOT working?

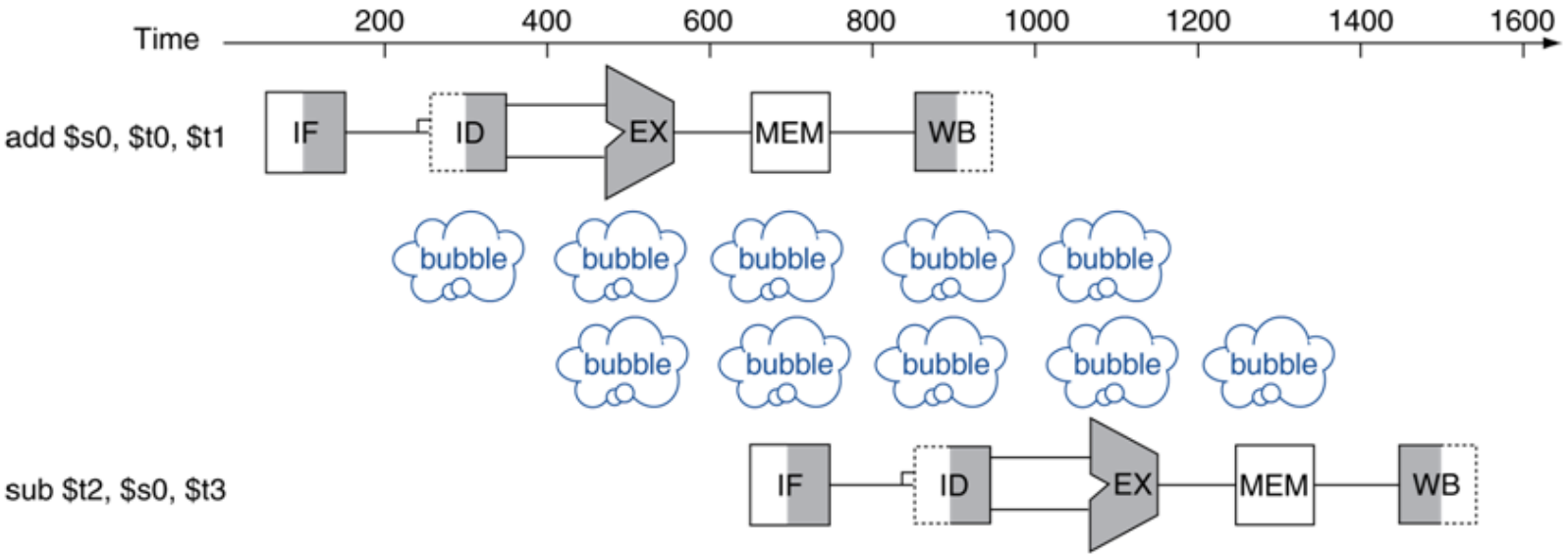
- Structure hazards
  - A required resource is busy
    E.g., instruction memory v.s. data memory
- Data hazard
  - Need to wait for previous instruction to complete its data read/write
- Control hazard
  - Deciding on control action depends on previous instruction
    E.g., beq

Structure Hazards

- Conflict for use of a resource
- In MIPS pipeline with a single memory
  - Load/store requires data access
  - Instruction fetch would have to stall for that cycle
    - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories
  - Or separate instruction/data caches

Data Hazards

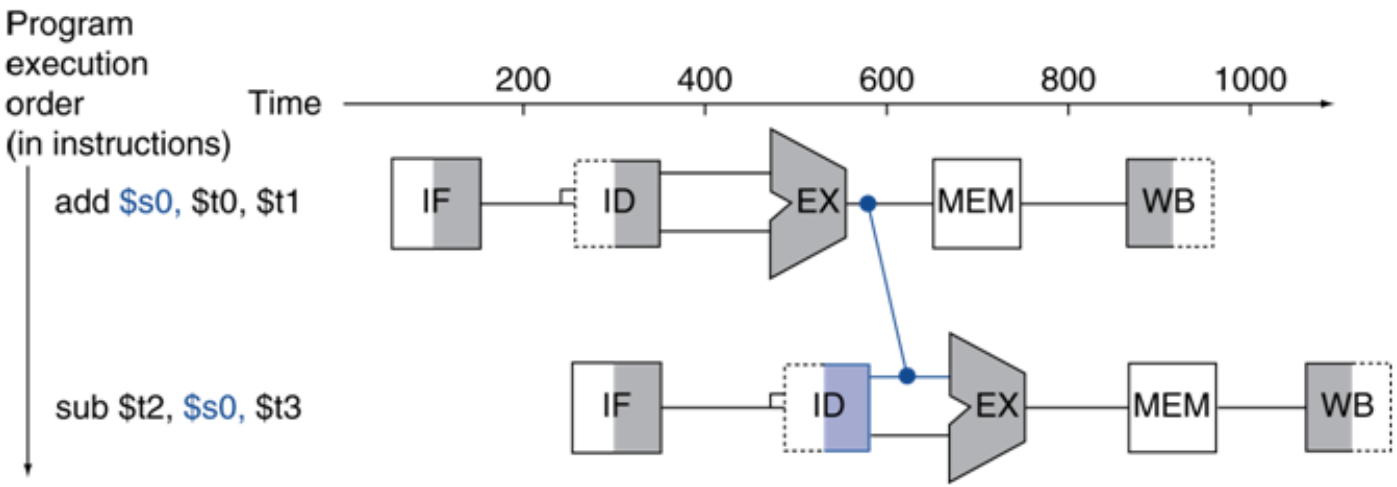
- An instruction depends on completion of data access by a previous instruction
  - add $s0, $t0, $t1
    sub $t2, $s0, $t3



Forwarding (aka Bypassing)

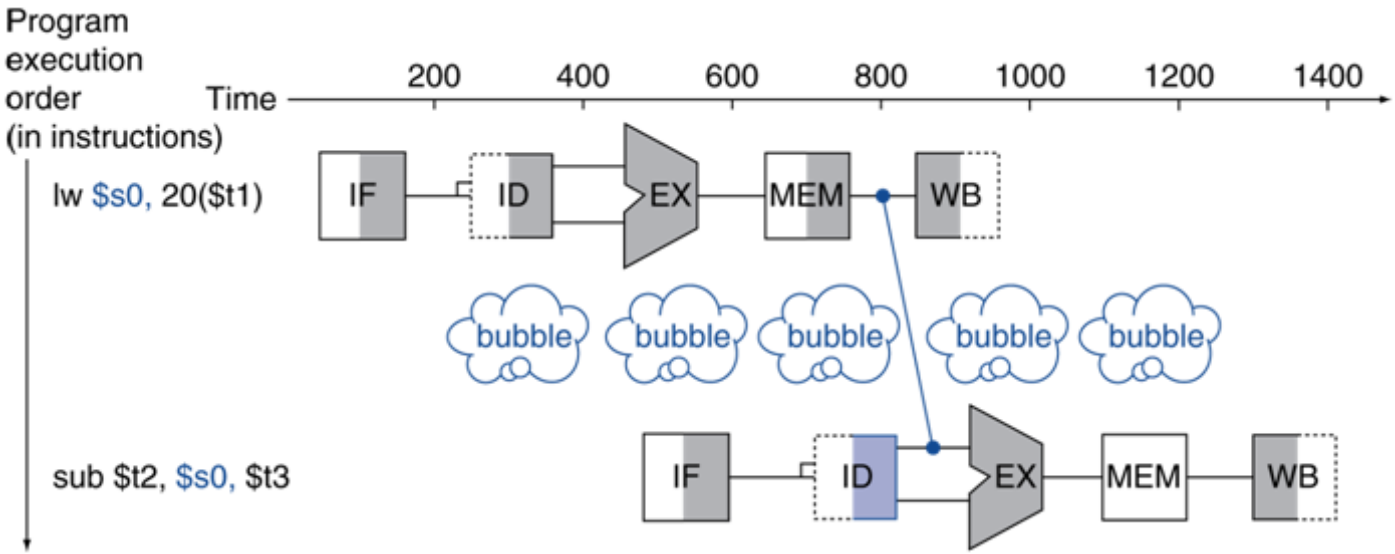
Use result when it is computed

- Don't wait for it to be stored in a register
- Requires extra connections in the datapath



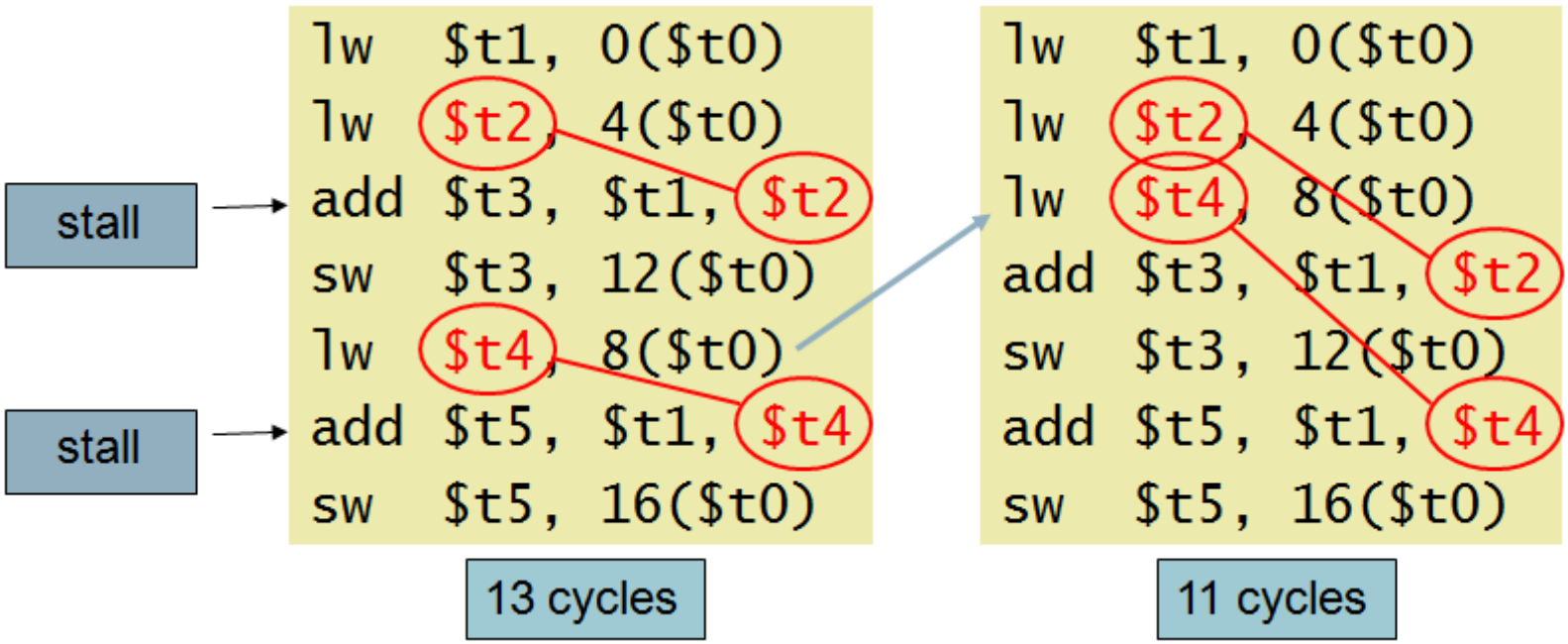
Load-Use Data Hazards

- Can't always avoid stalls by forwarding
- If value not computed when needed
- Can't forward backward in time!



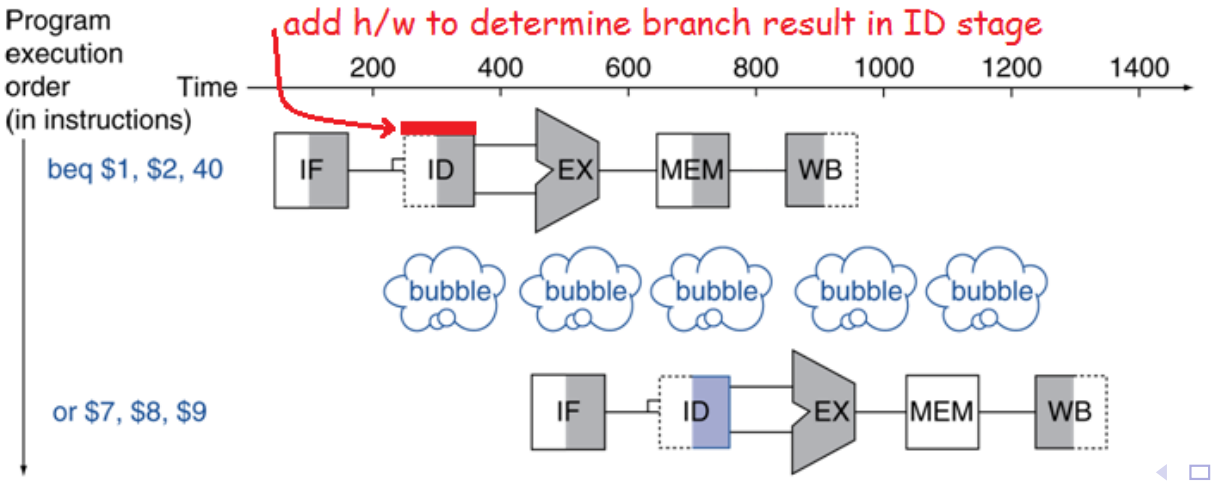
Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for A = B + E; C = B + F;



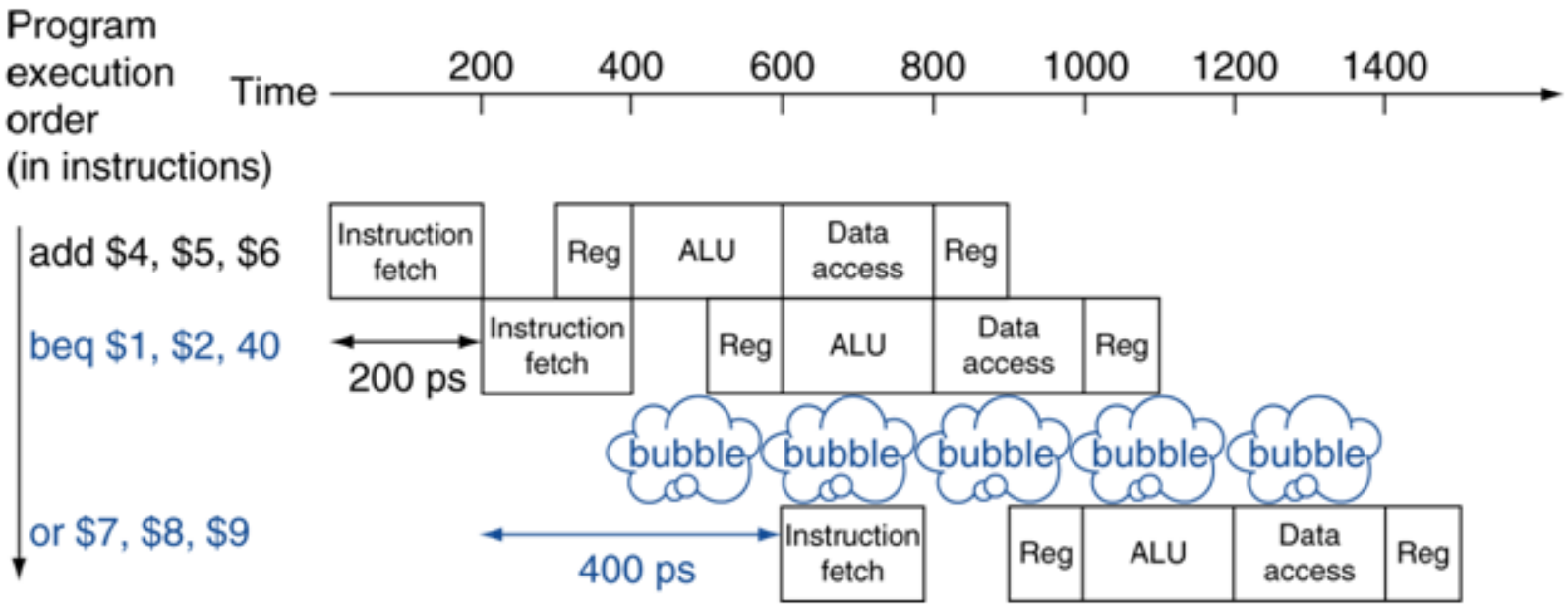
Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline can't always fetch correct instruction
    - Still working on ID stage of branch
- In MIPS pipeline
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in ID stage



Stall on Branch

Wait until branch outcome determined before fetching next instruction

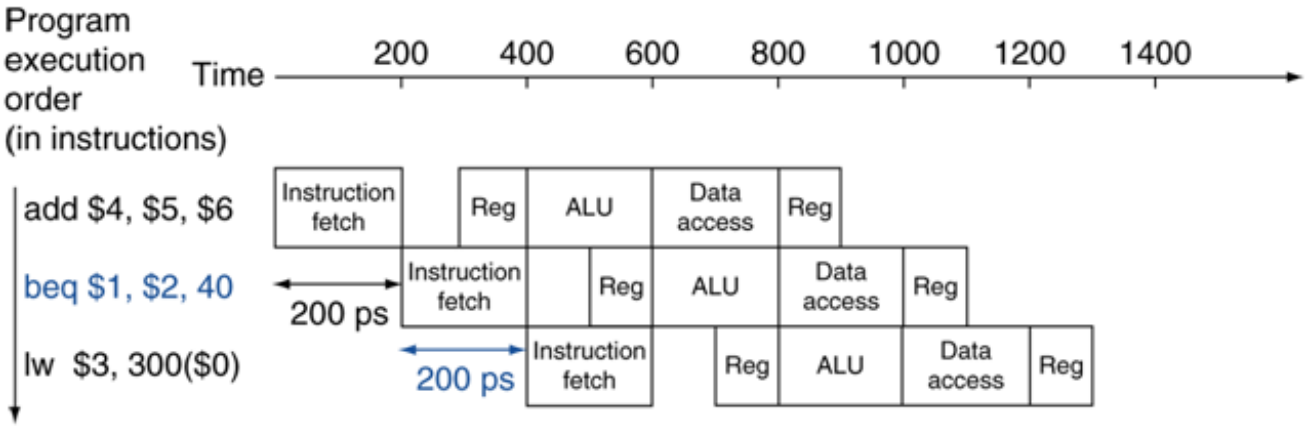


Branch Prediction

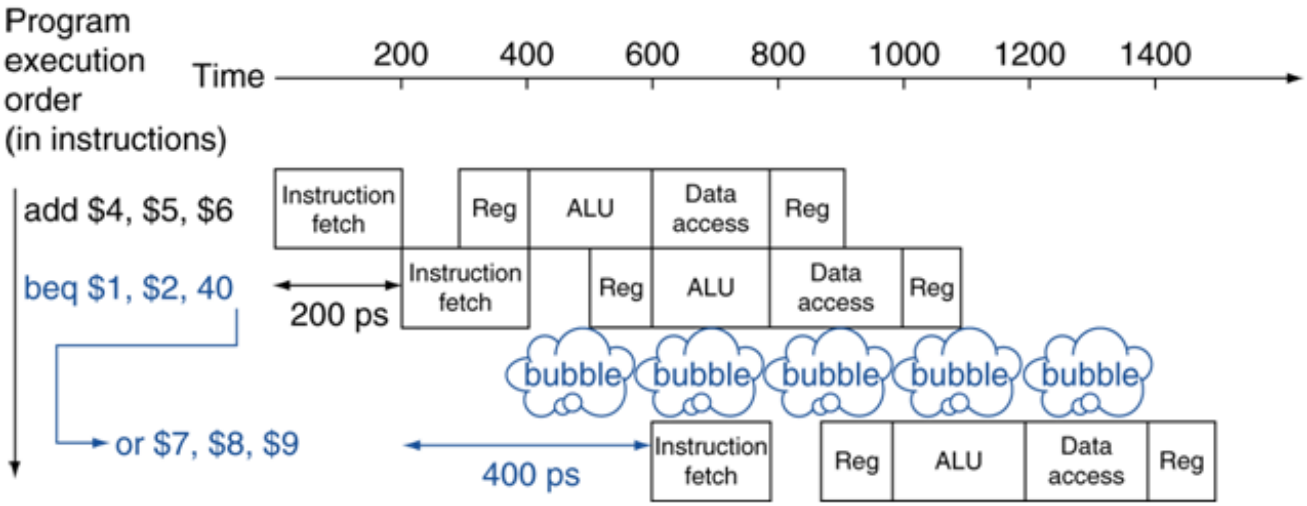
- Longer pipelines can't readily determine branch outcome early
  - Stall penalty becomes unacceptable
- Predict outcome of branch
  - Only stall if prediction is wrong
- In MIPS pipeline
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

MIPS with Predict Not Taken

Prediction correct



Prediction incorrect



More-Realistic Branch Prediction

Static branch prediction

- Based on typical branch behavior
- Example: loop and if-statement branches
  - Predict backward branches taken
  - Predict forward branches not taken

```
x = 1000;
for(i=0;i<10;i++){
   x = x * 1.05;
}
```

```
x = 1000;
i = 0;
do{
   x = x * 1.05;
   i++;
}while(x < 10000);
```

```
x = 1000;
i = 0;
while(x < 10000){
   x = x * 1.05;
   i++;
}
```

More-Realistic Branch Prediction

Dynamic branch prediction

- Hardware measures actual branch behavior

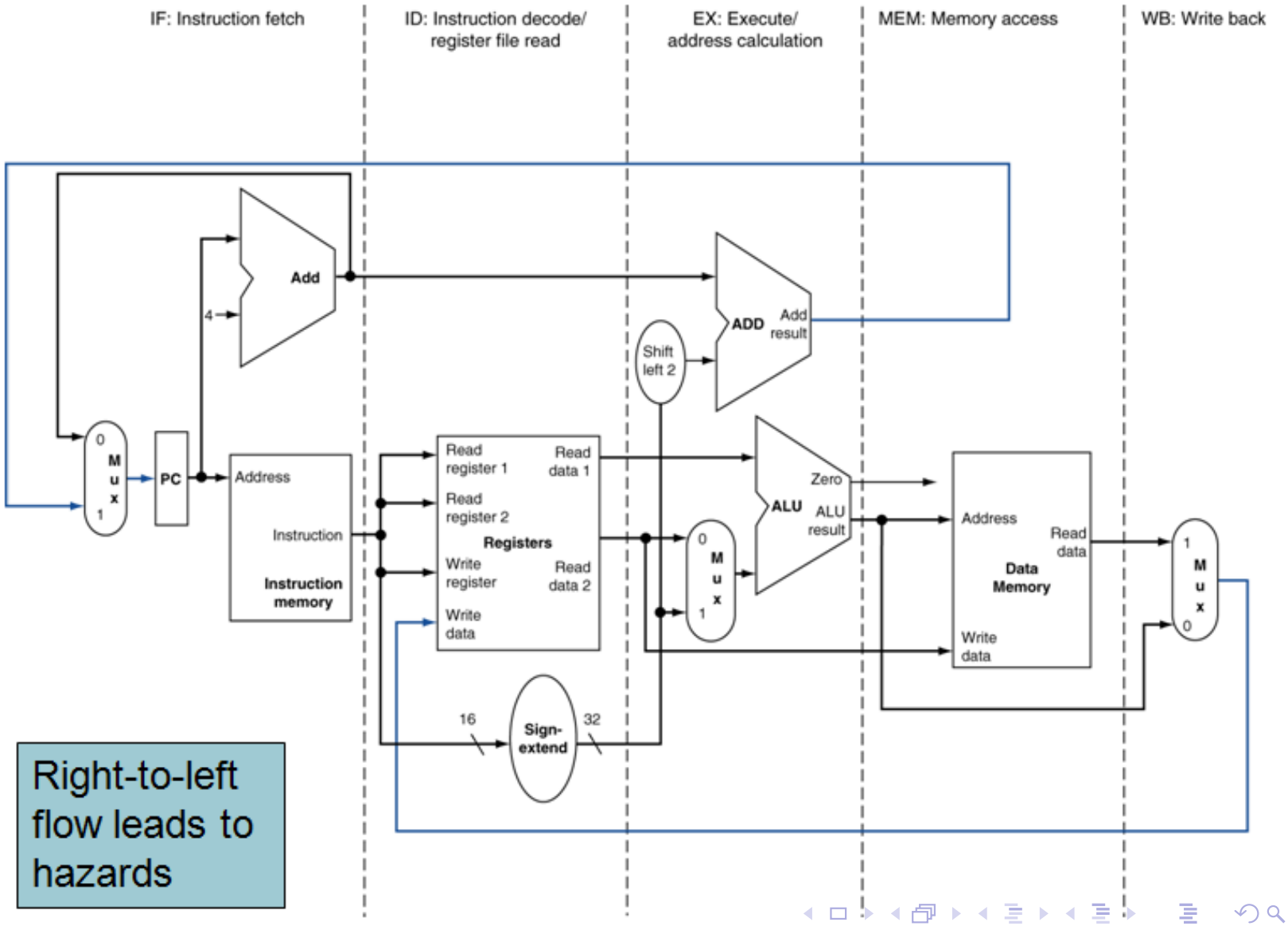
  E.g., record recent history of each branch
- Assume future behavior will continue the trend

  $\Rightarrow$ When wrong, stall while re-fetching, and update history

Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

MIPS Pipelined Datapath

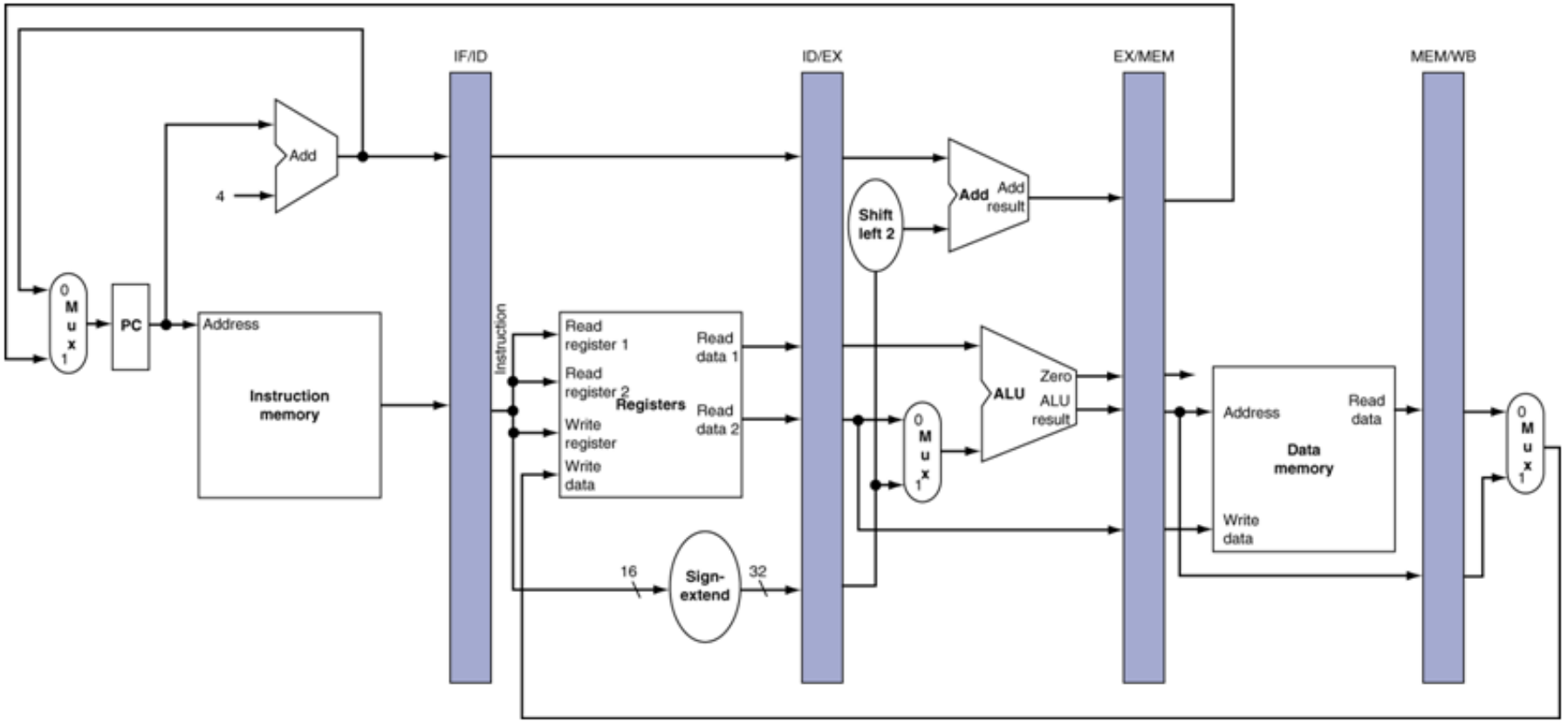


Right-to-left flow leads to hazards

Pipeline registers

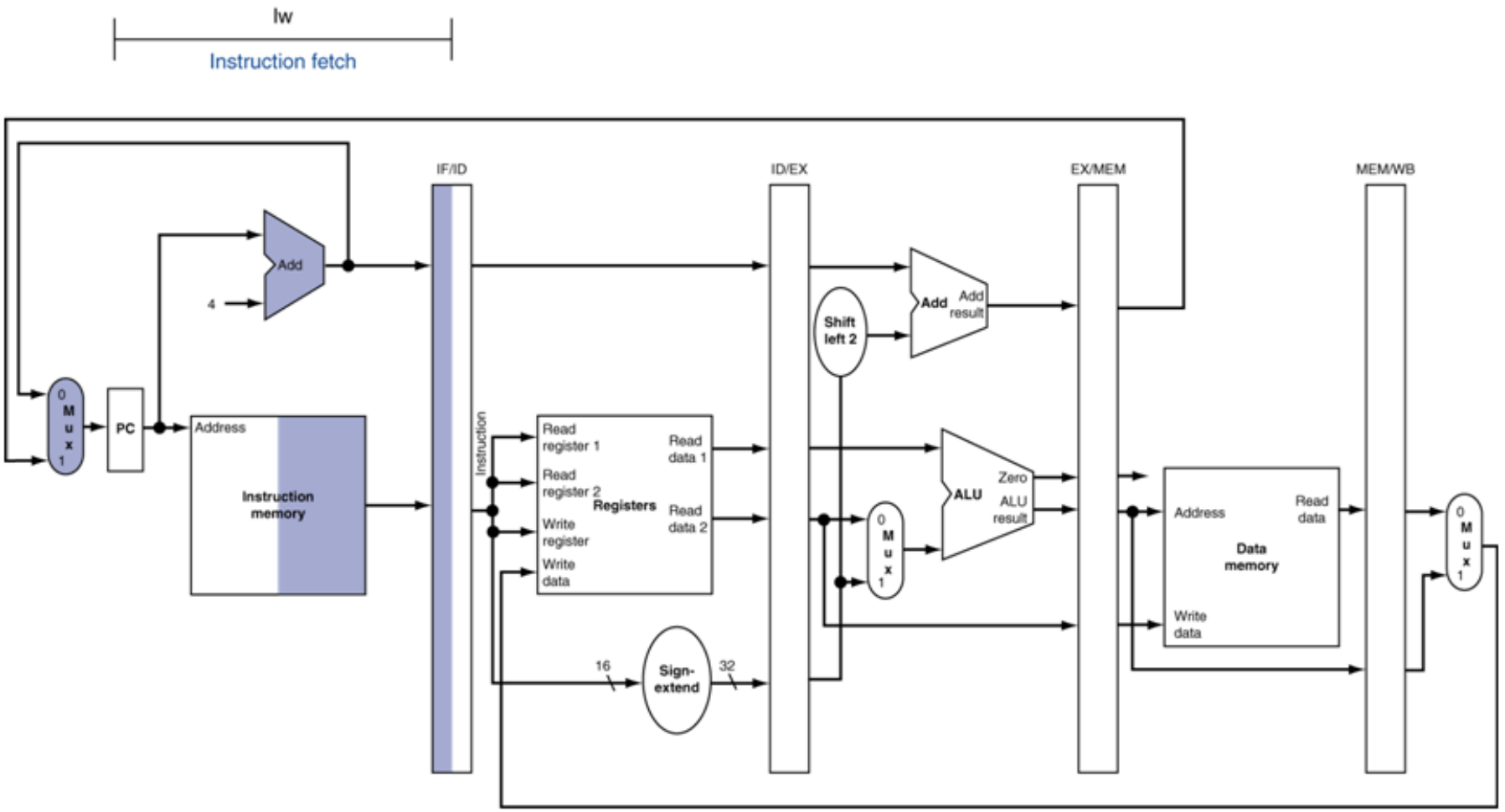
- Need registers between stages

  $\Rightarrow$ To hold information produced in previous cycle

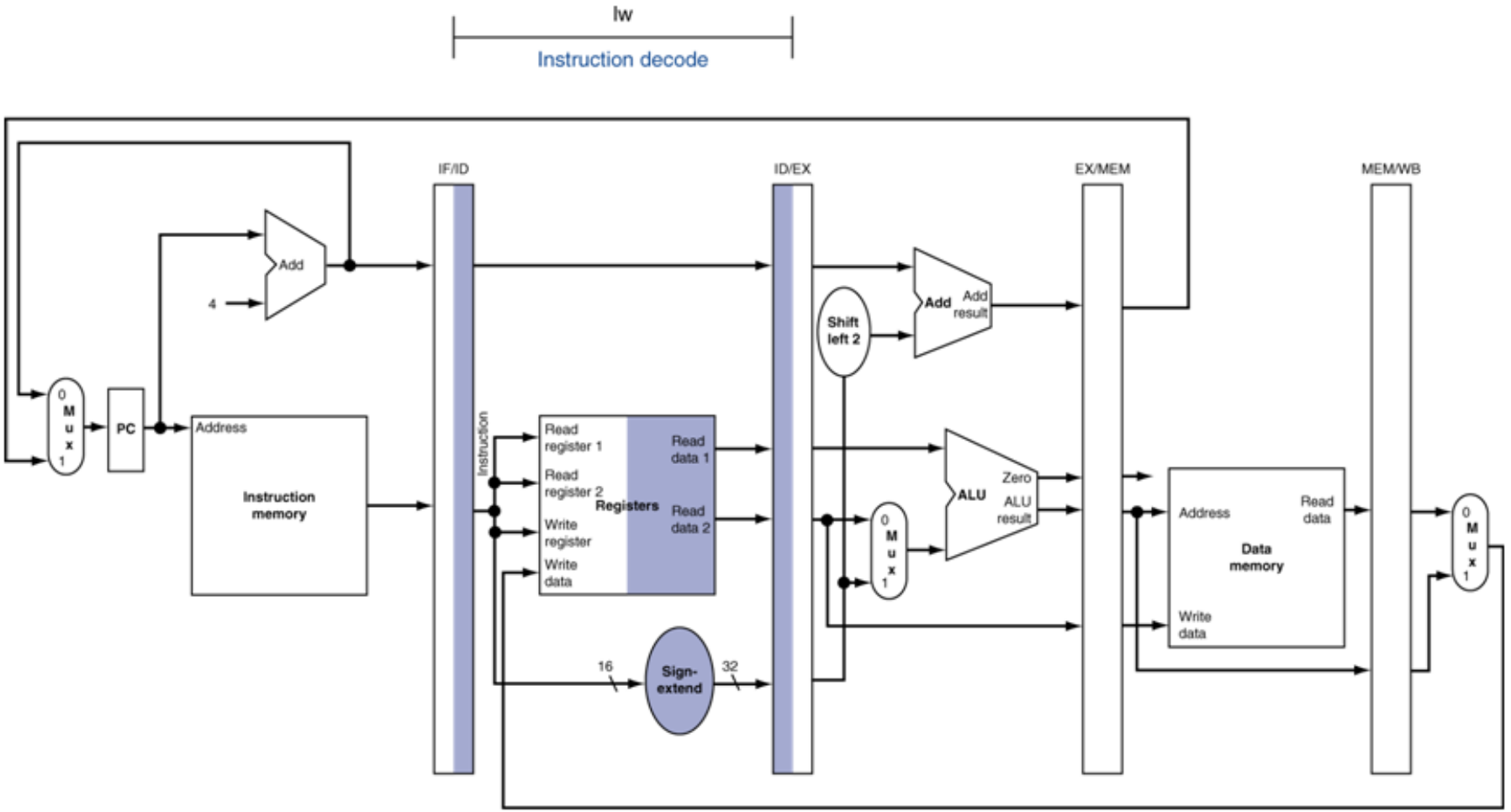


Note: no extra h/w to do branch in ID stage.

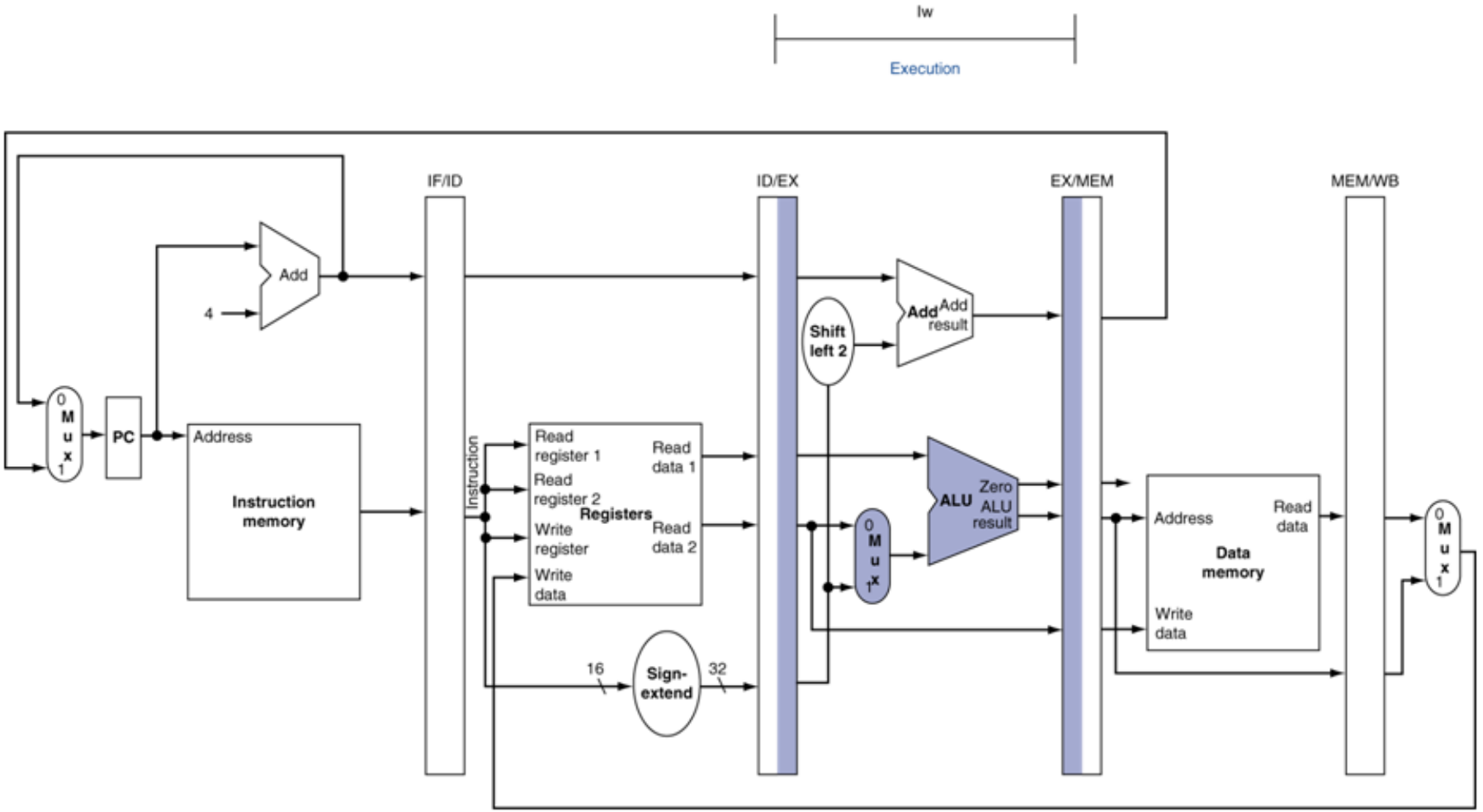
Instruction Fetch



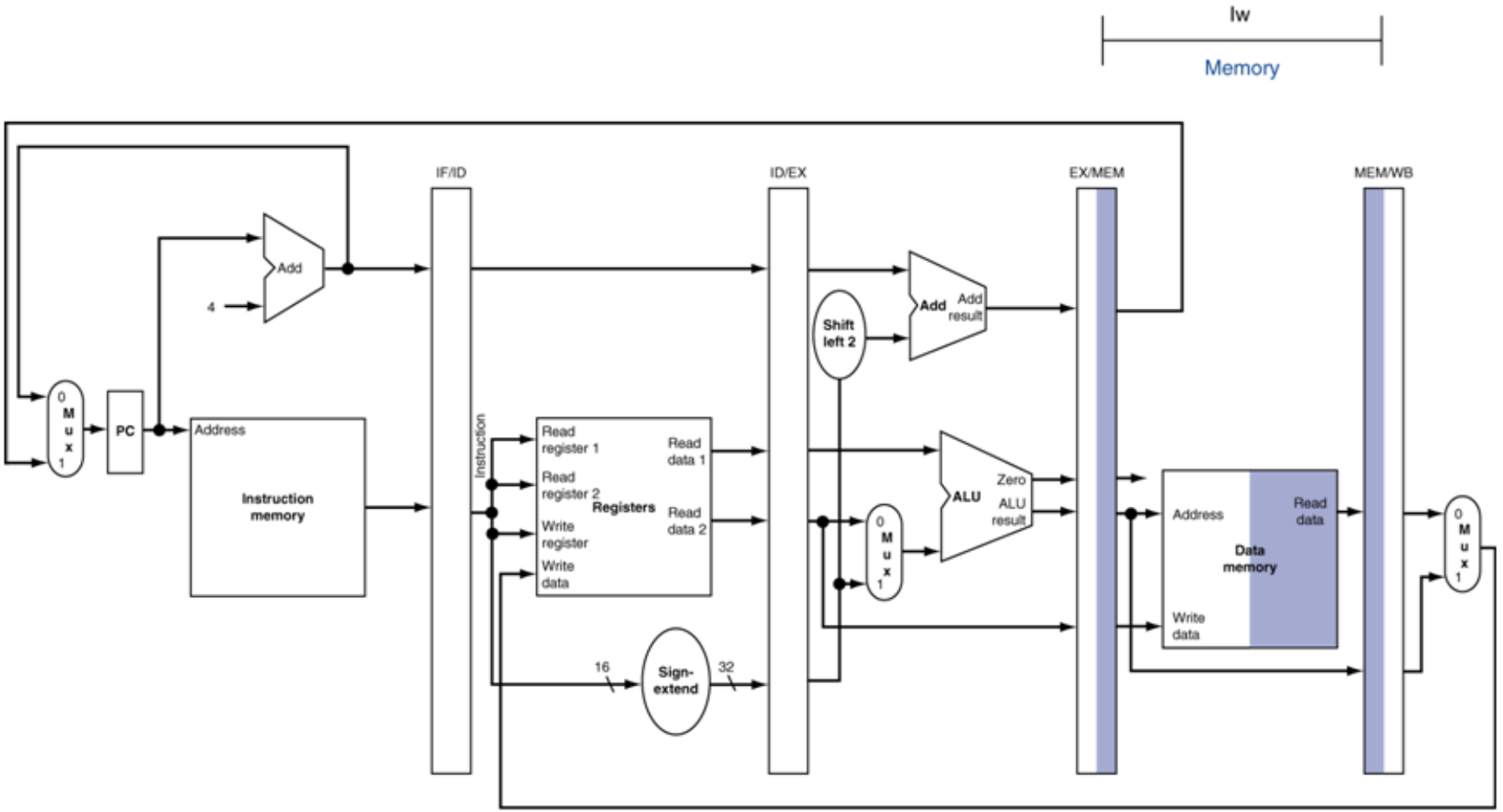
Instruction Decode



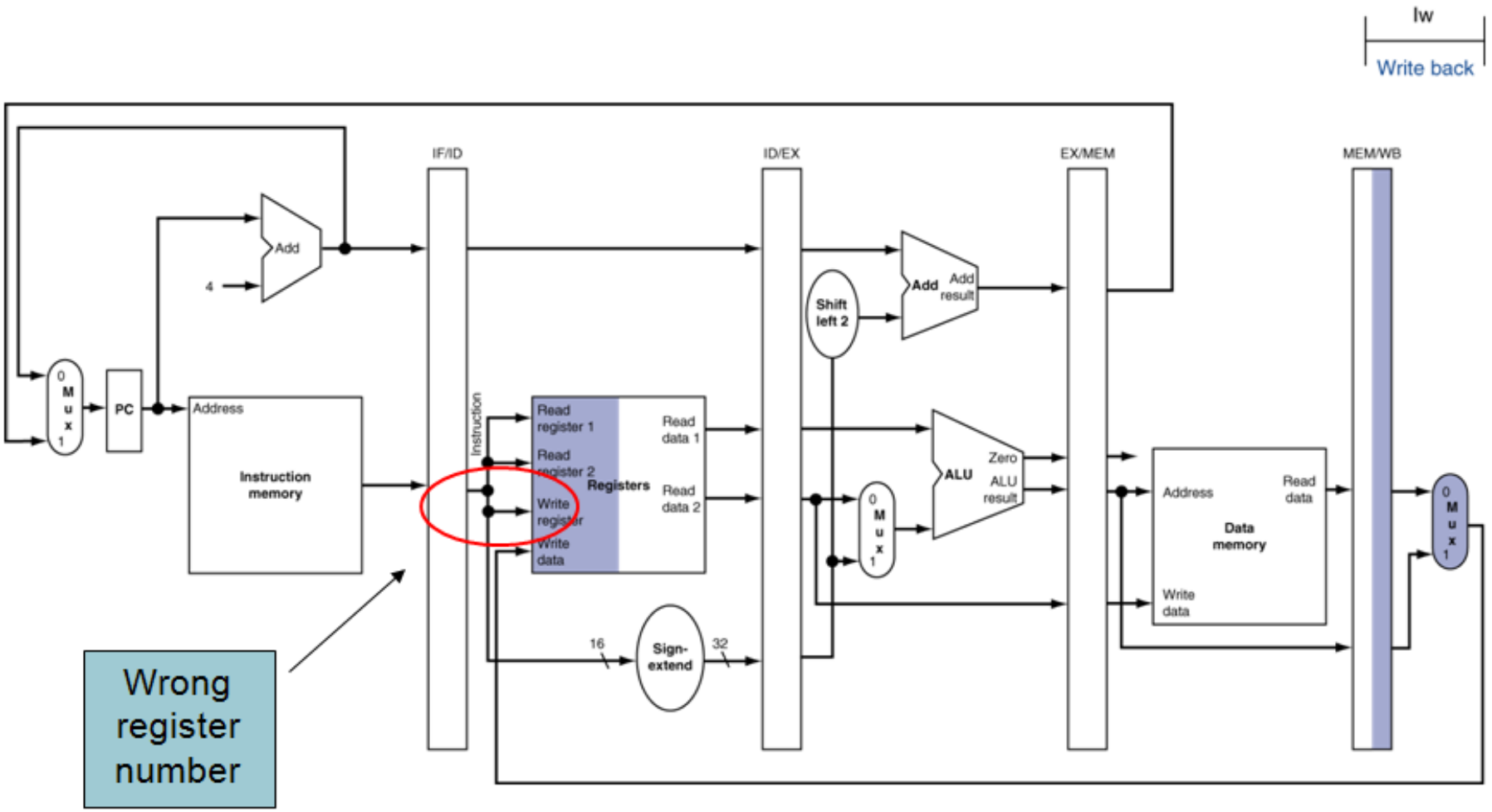
Execution for Load



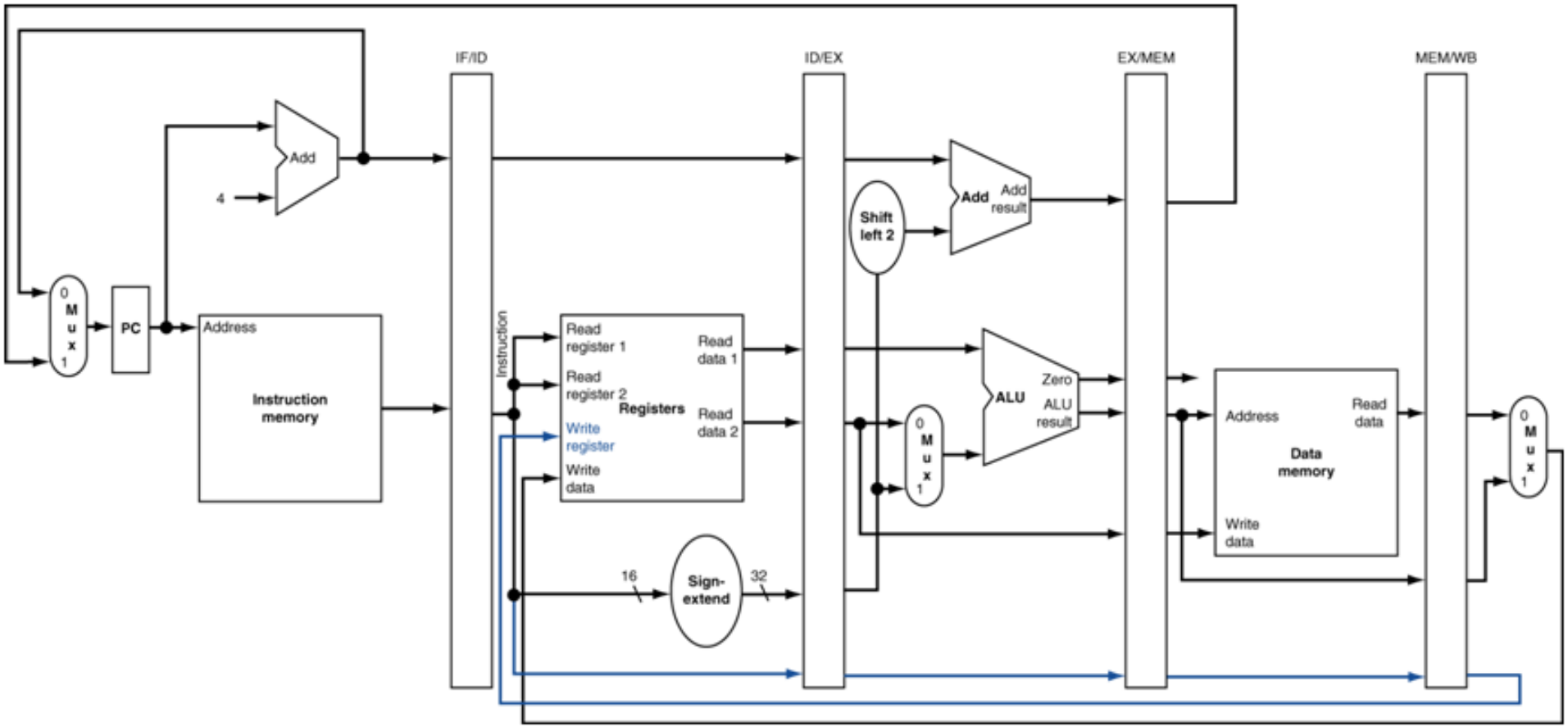
Memory Access for Load



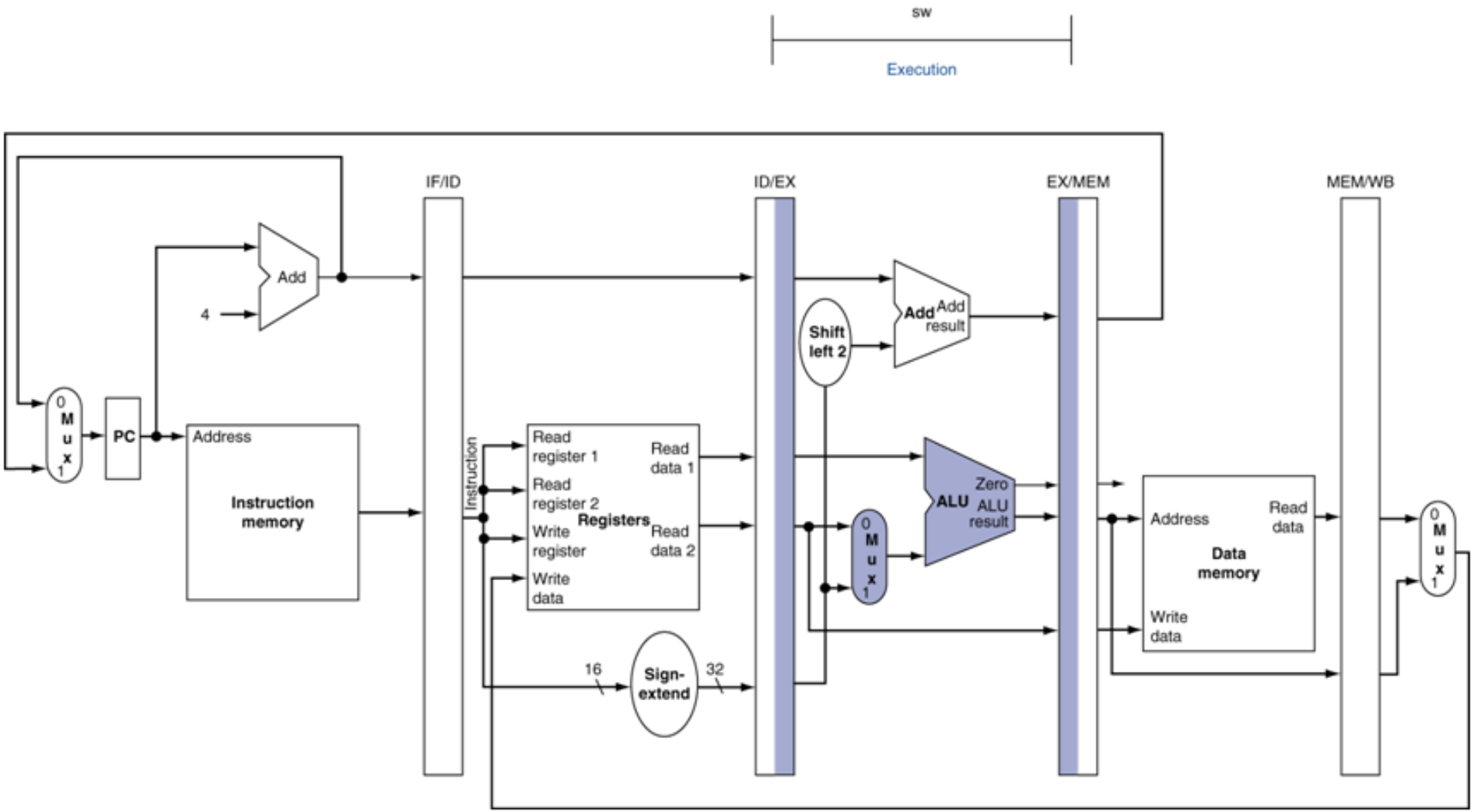
Write Back Stage for Load



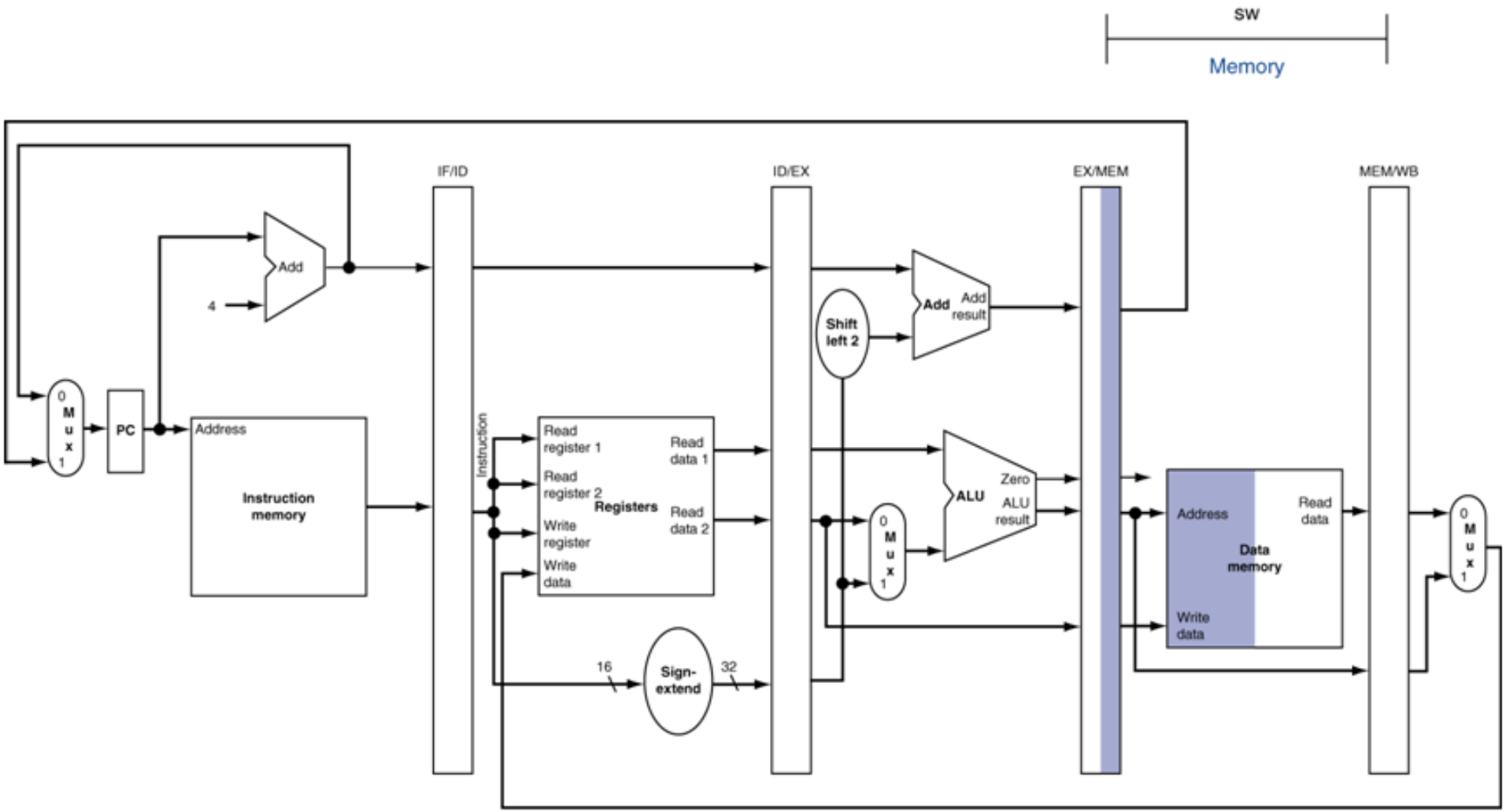
Corrected Datapath for Load



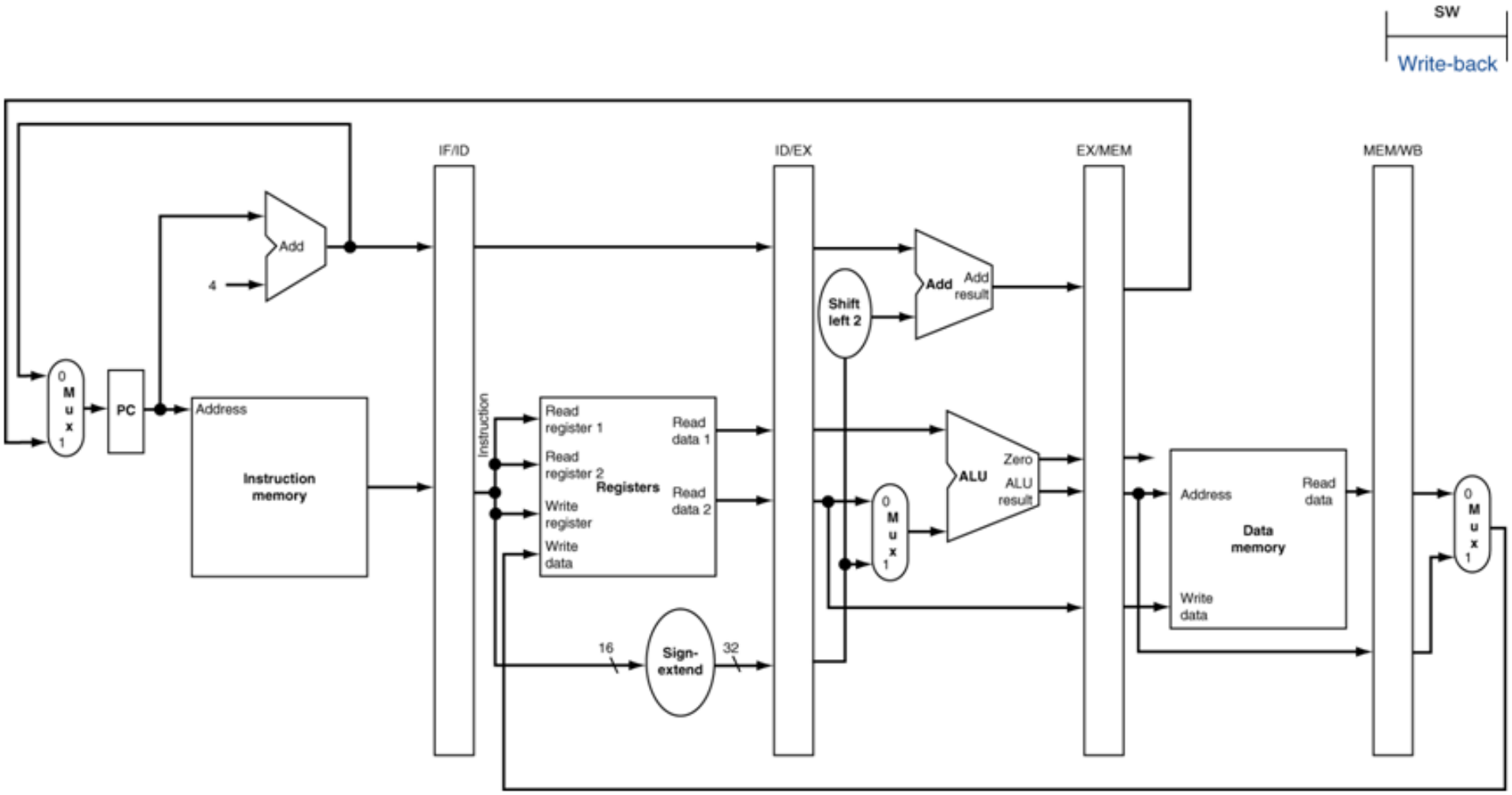
EX for Store


sw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data
16
Sign-extend
32

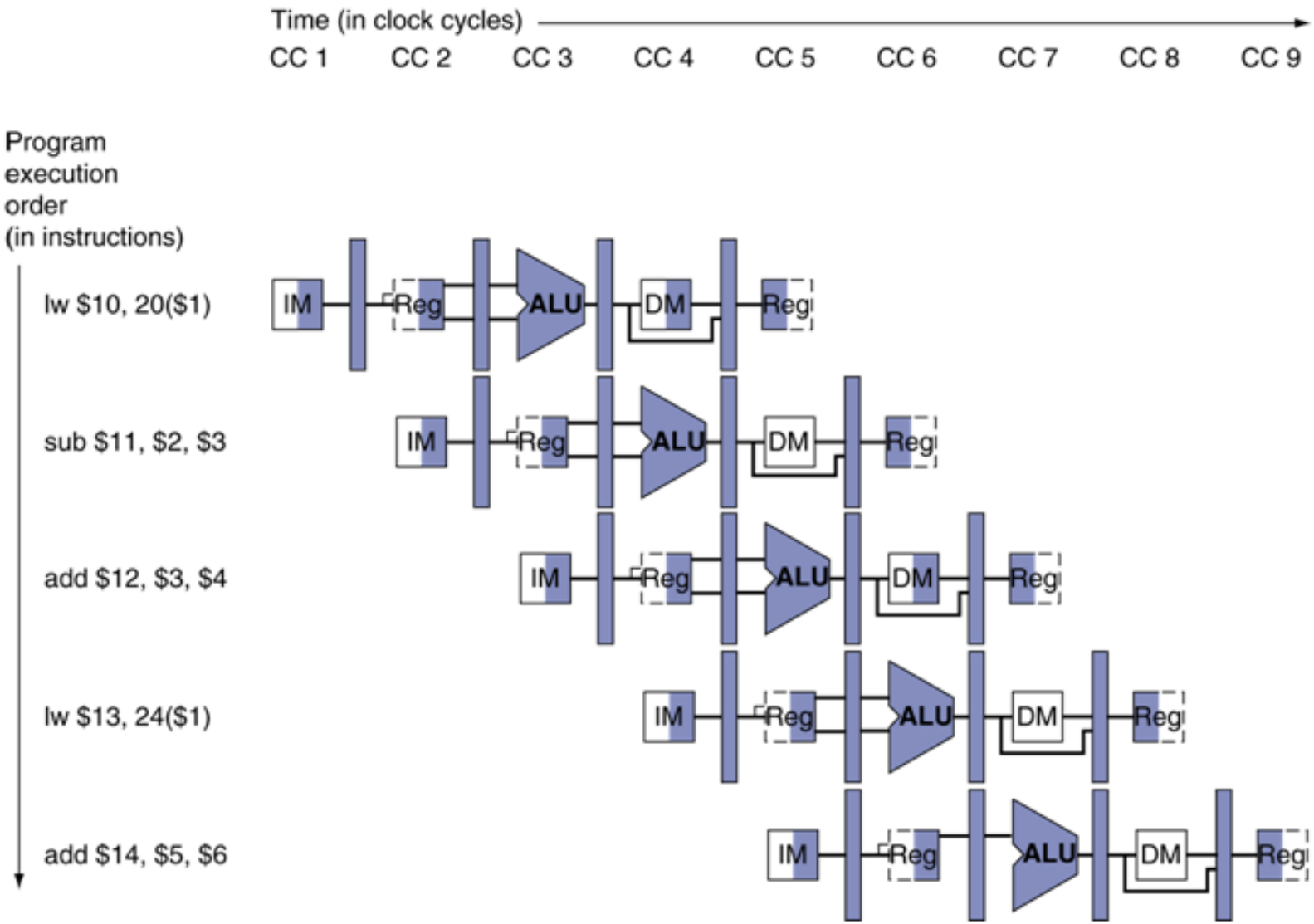
MEM for Store



WB for Store

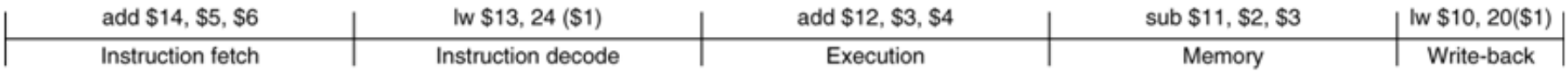

SW
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
0
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign-extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
Address
Read data
Data memory
Write data

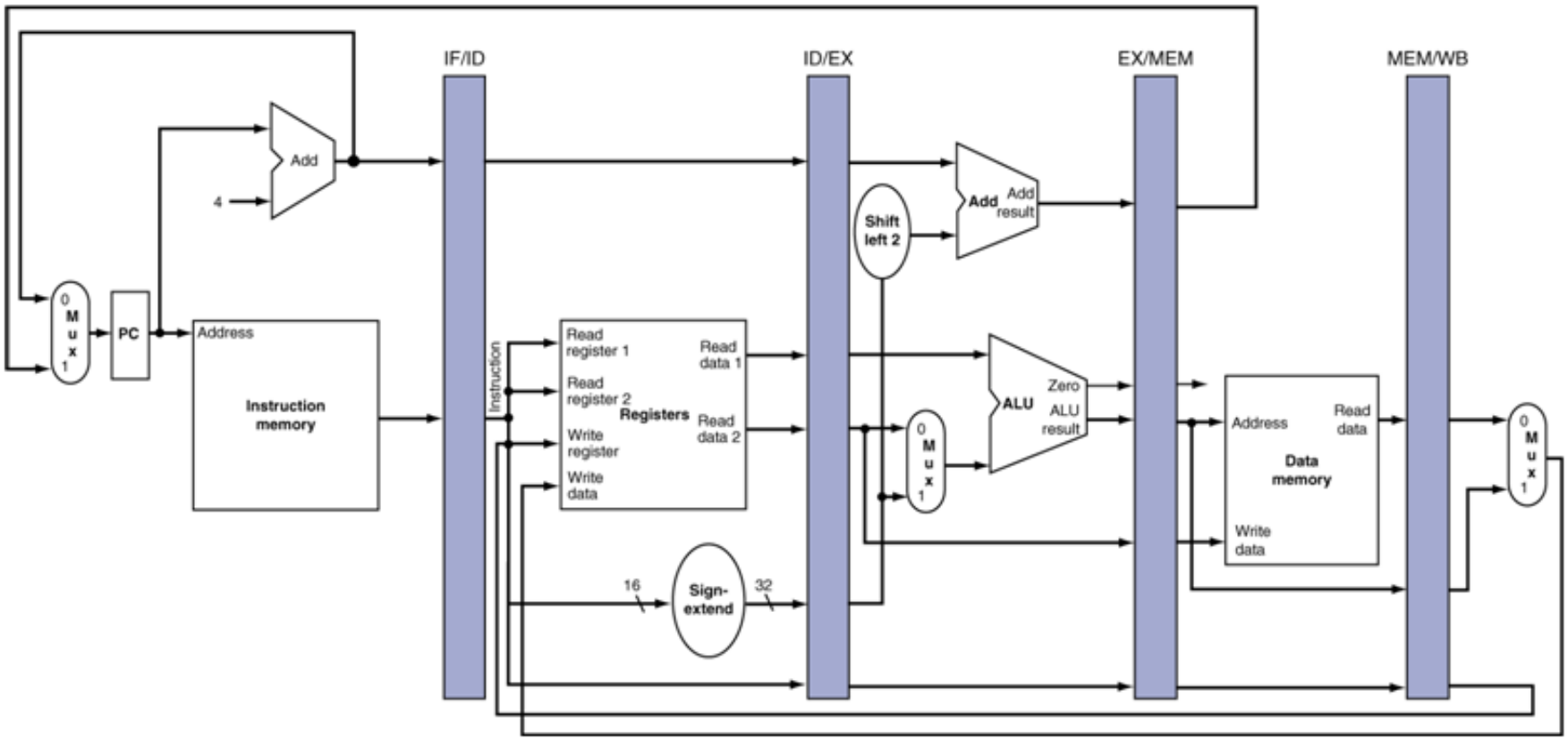
Multi-Cycle Pipeline Diagram: Resource Usage



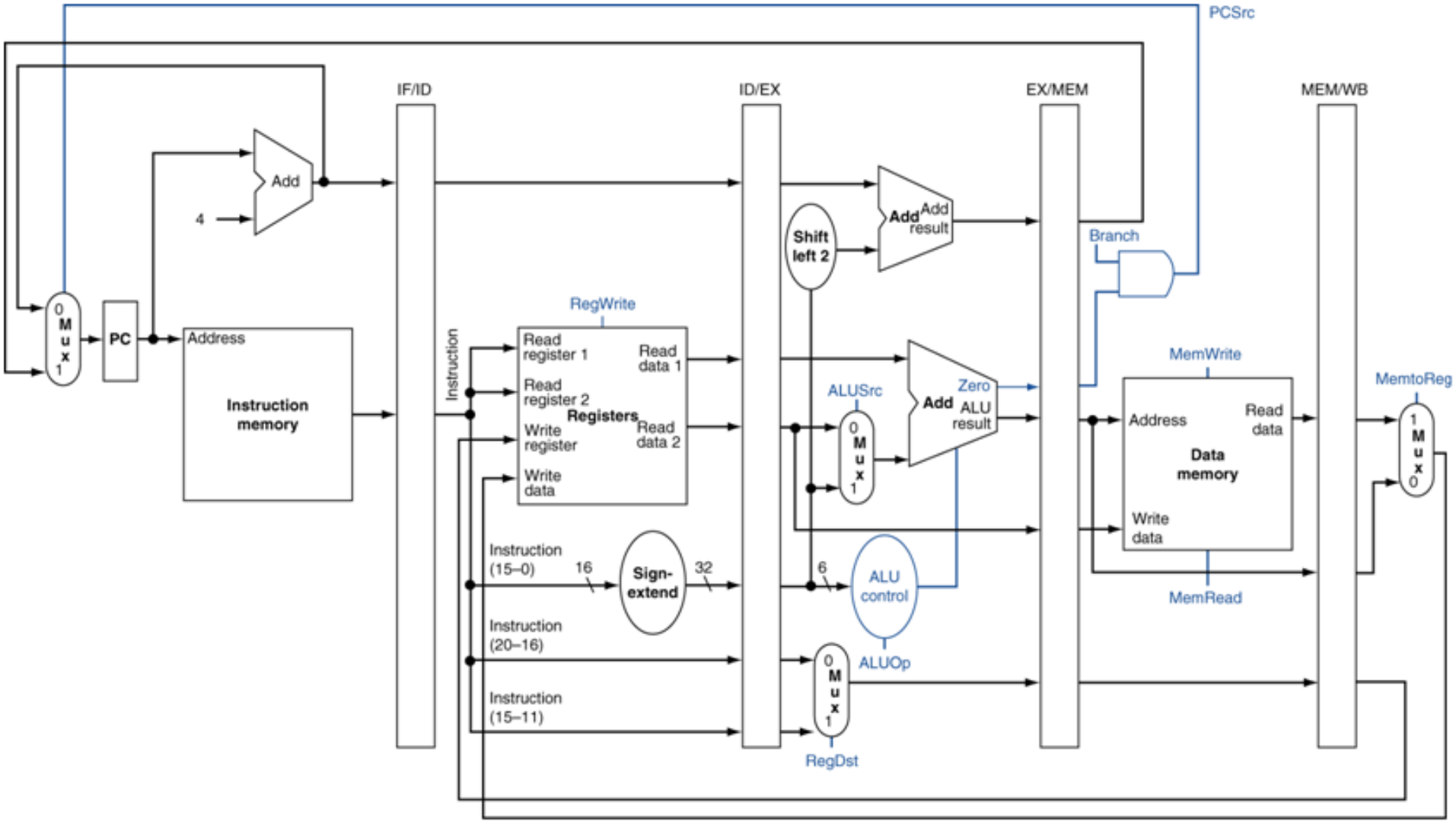
Single-Cycle Pipeline Diagram:

State of pipeline in a given cycle



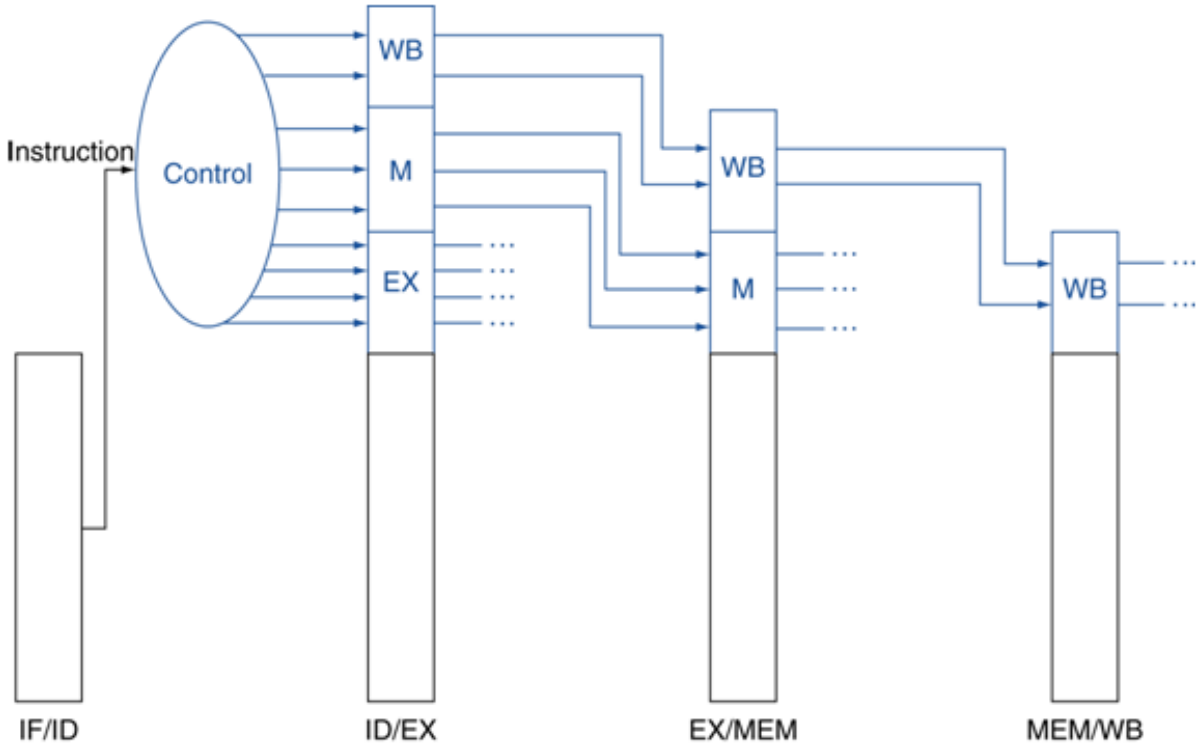


Pipelined Control (Simplified)

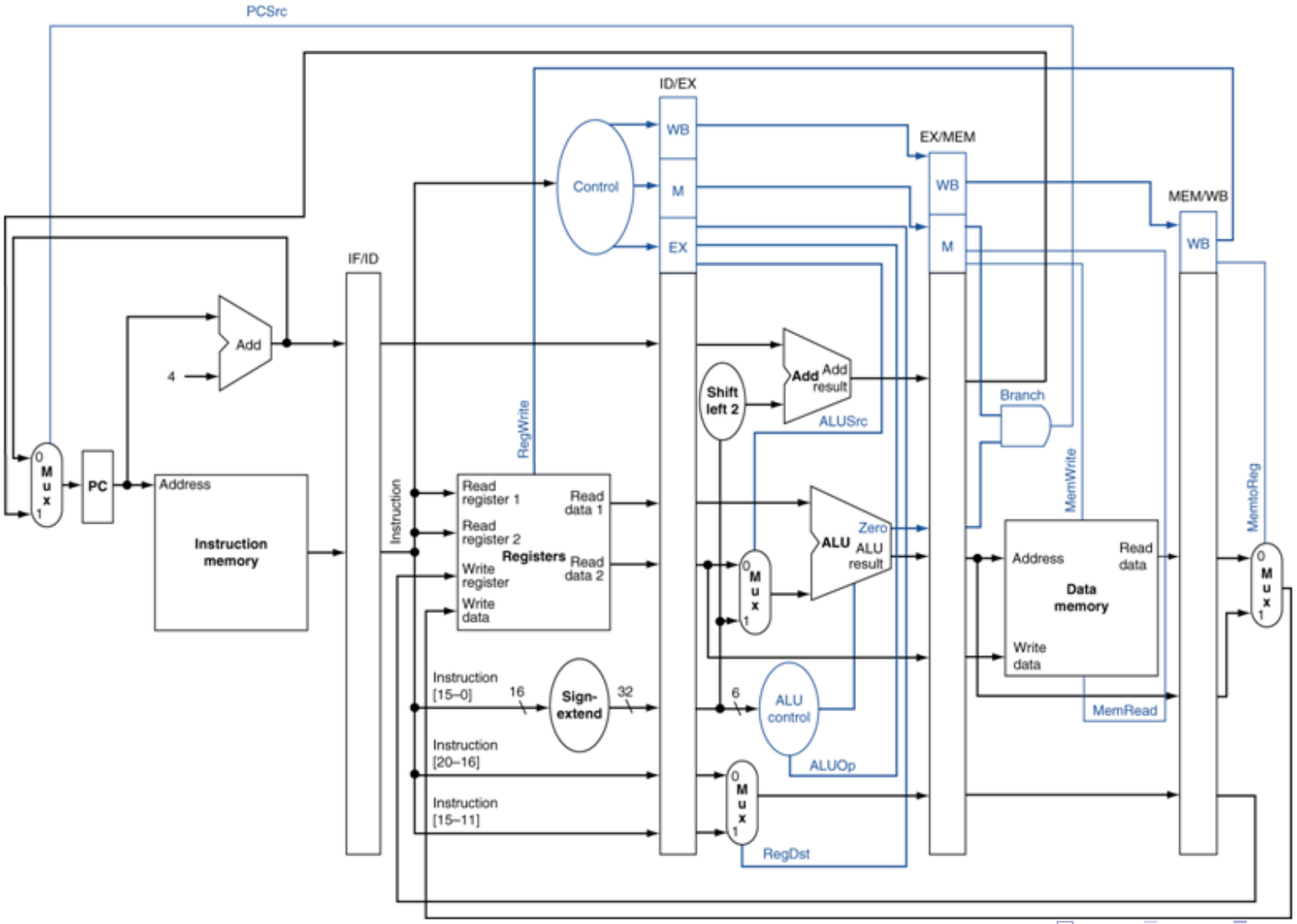


Pipelined Control

Control signals derived from instruction



Pipelined Control



Data Hazards in ALU Instructions

Consider this code sequence:

sub $2, $1,$3

and $12,$2,$5

or $13,$6,$2

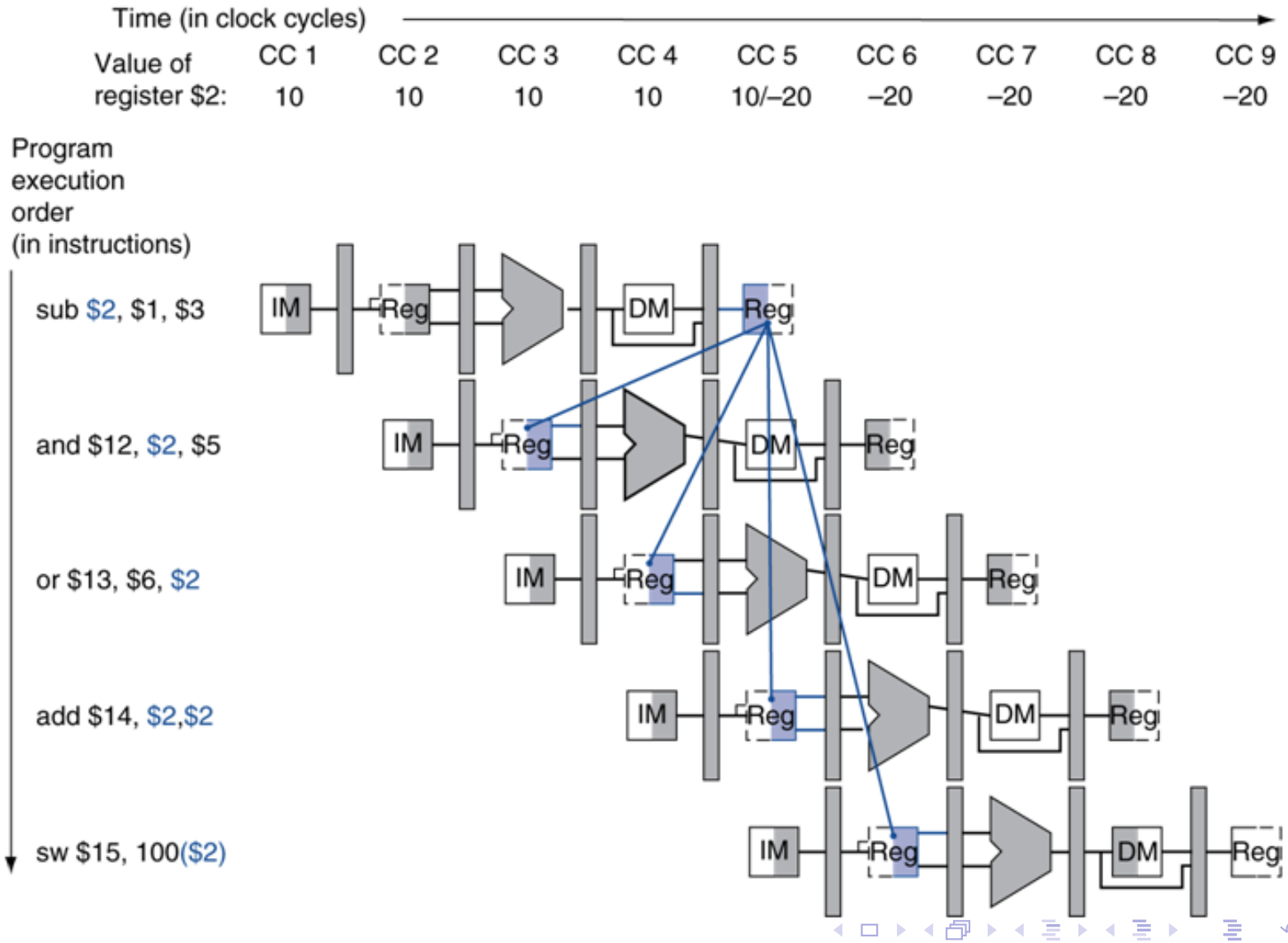
add $14,$2,$2

sw $15,100($2)

We can resolve hazards with forwarding

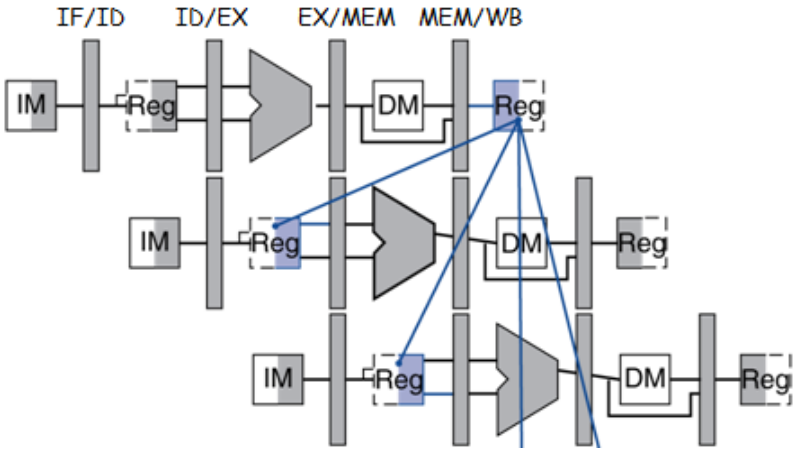
$\Rightarrow$ How do we detect when to forward?

Dependencies & Forwarding



Detecting the Need to Forward

- Pass register numbers along pipeline
- ALU operand register numbers in EX stage are given by
  - ID/EX.RegisterRs, ID/EX.RegisterRt



Data hazards when

(1a) EX/MEM.RegisterRd = ID/EX.RegisterRs

(1b) EX/MEM.RegisterRd = ID/EX.RegisterRt

(2a) MEM/WB.RegisterRd = ID/EX.RegisterRs

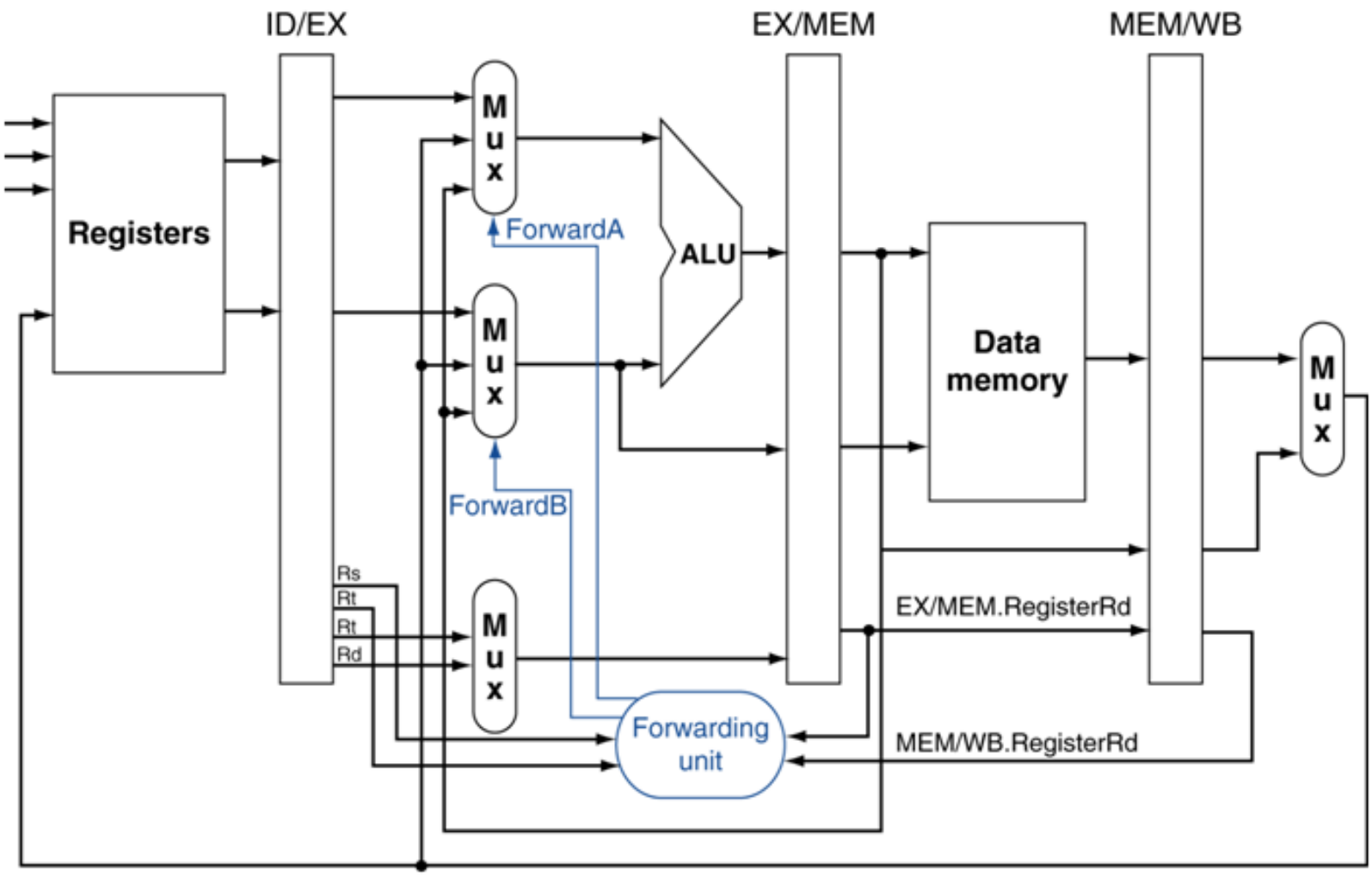
(2b) MEM/WB.RegisterRd = ID/EX.RegisterRt

- (1a - 1b) Forward from EX/MEM pipeline register
- (2a - 2b) Forward from MEM/WB pipeline register

Detecting the Need to Forward

- But only if forwarding instruction will write to a register!
  - EX/MEM.RegWrite, MEM/WB.RegWrite
- And only if Rd for that instruction is not $zero
  $\Rightarrow$ nop: 0x00000000
  - EX/MEM.RegisterRd $\neq 0$
  - MEM/WB.RegisterRd $\neq 0$

Forwarding Paths



Forwarding Conditions

- EX hazard
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq 0$) and (EX/MEM.RegisterRd $=$ ID/EX.RegisterRs)) $\Rightarrow$ ForwardA = [10]
  - if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq 0$) and (EX/MEM.RegisterRd $=$ ID/EX.RegisterRt)) $\Rightarrow$ ForwardB = [10]
- MEM hazard
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq 0$) and (MEM/WB.RegisterRd $=$ ID/EX.RegisterRs)) $\Rightarrow$ ForwardA = [01]
  - if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq 0$) and (MEM/WB.RegisterRd $=$ ID/EX.RegisterRt)) $\Rightarrow$ ForwardB = [01]

Forwarding Conflict?

Instr #	Instruction	rs	rt	rd
1	add $t0, $s1, $s2	17	18	8
2	add $t0, $t0, $s3	8	19	8
3	add $t0, $t0, $s4	8	20	8

cc1	cc2	cc3
IF/ID	ID/EX	EX/MEM.rd = 8
	IF/ID	ID/EX.rs = 8
		IF/ID
		Forward
		from EX/MEM

- At cc3, EX/MEM.rd = 8 (instr. 1) and ID/EX.rs = 8 (instr. 2) $\Rightarrow$ EX hazard $\Rightarrow$ forward rs's value from EX/MEM

Forwarding Conflict?

Instr #	Instruction	rs	rt	rd	cc3	cc4	cc5	cc6
1	add $t0, $s1, $s2	17	18	8	EX/MEM.rd = 8	MEM/WB.rd = 8	done	
2	add $t0, $t0, $s3	8	19	8	ID/EX.rs = 8	EX/MEM.rd = 8	MEM/WB.rd = 8	done
3	add $t0, $t0, $s4	8	20	8	IF/ID	ID/EX.rs = 8	EX/MEM.rd = 8	MEM/WB.rd = 8
					Forward	Forward		
					from EX/MEM	from ?		

- At cc4, EX/MEM.rd = 8 (instr. 2) and ID/EX.rs = 8 (instr. 3)
- Also, MEM/WB.rd = 8 (instr. 1) and ID/EX.rs = 8 (instr. 3)
- $\Rightarrow$ "Double Data Hazard"
- Which one should we forward from?

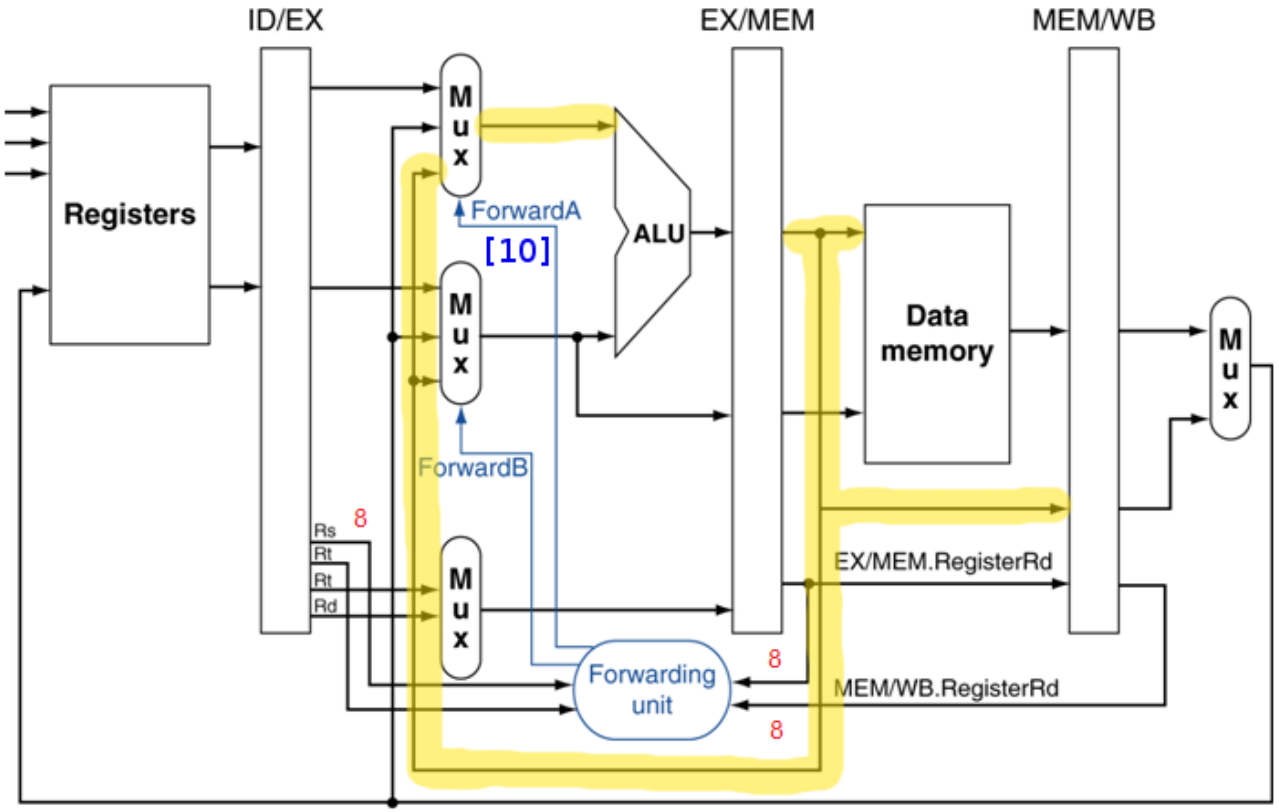
Forwarding Conflict?

Instr #	Instruction	rs	rt	rd	cc3	cc4	cc5	cc6
1	add $t0, $s1, $s2	17	18	8	EX/MEM.rd = 8	MEM/WB.rd = 8	done	
2	add $t0, $t0, $s3	8	19	8	ID/EX.rs = 8	EX/MEM.rd = 8	MEM/WB.rd = 8	done
3	add $t0, $t0, $s4	8	20	8	IF/ID	ID/EX.rs = 8	EX/MEM.rd = 8	MEM/WB.rd = 8
					Forward	Forward		
					from EX/MEM	from ?		

- At cc4, EX/MEM.rd = 8 (instr. 2) and ID/EX.rs = 8 (instr. 3)
- Also, MEM/WB.rd = 8 (instr. 1) and ID/EX.rs = 8 (instr. 3)
- $\Rightarrow$ "Double Data Hazard"
- Which one should we forward from?
  - if instr. 2 works correctly and it writes to rd, then rd from instr. 2 is fresher than rd from instr. 1.
    $\Rightarrow$ we should forward from EX/MEM (instr. 2).

Forwarding Conflict?

If both EX/MEM.registerRd and MEM/WB/registerRd match an ALU operand, choose the freshest one.

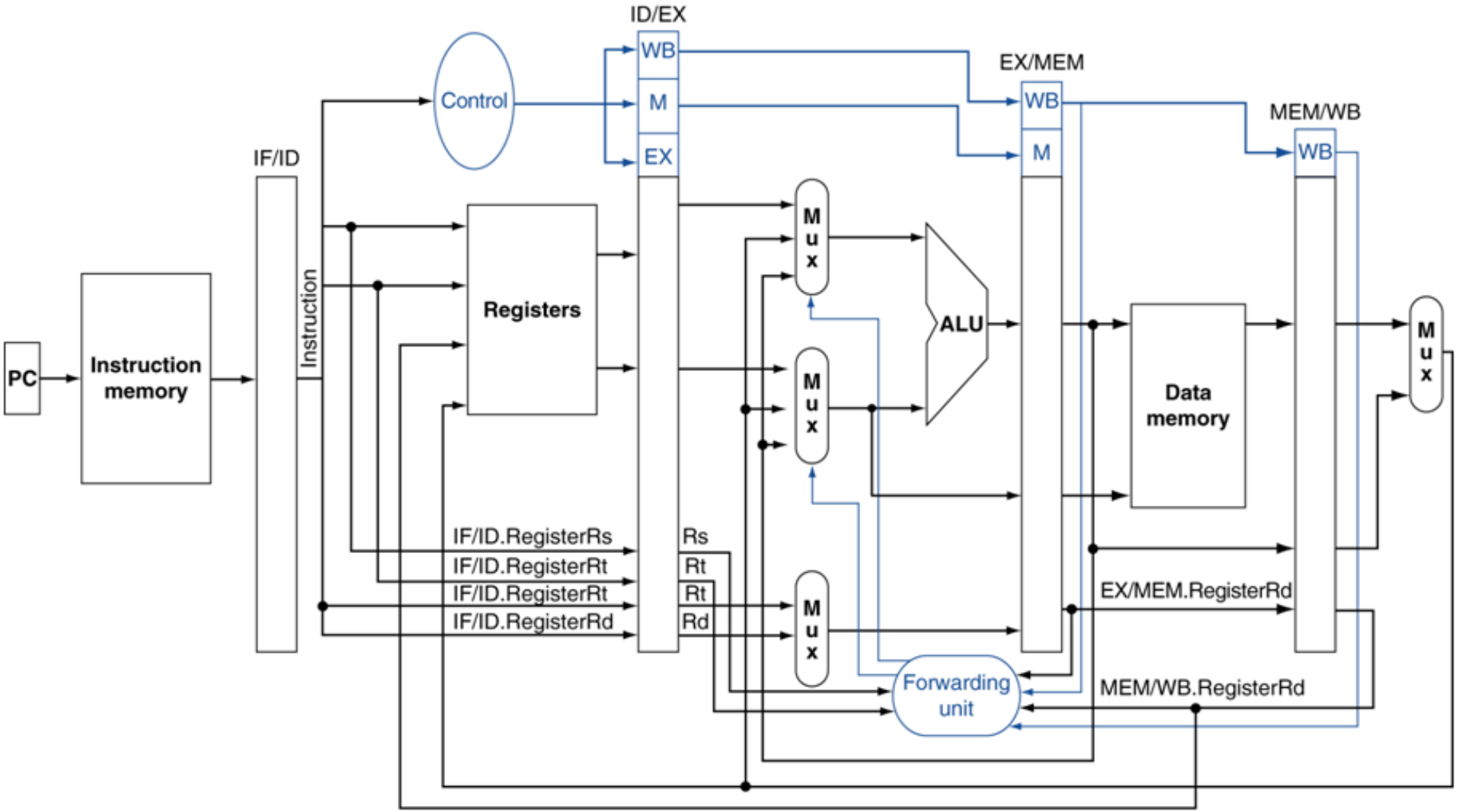


Revised Forwarding Condition

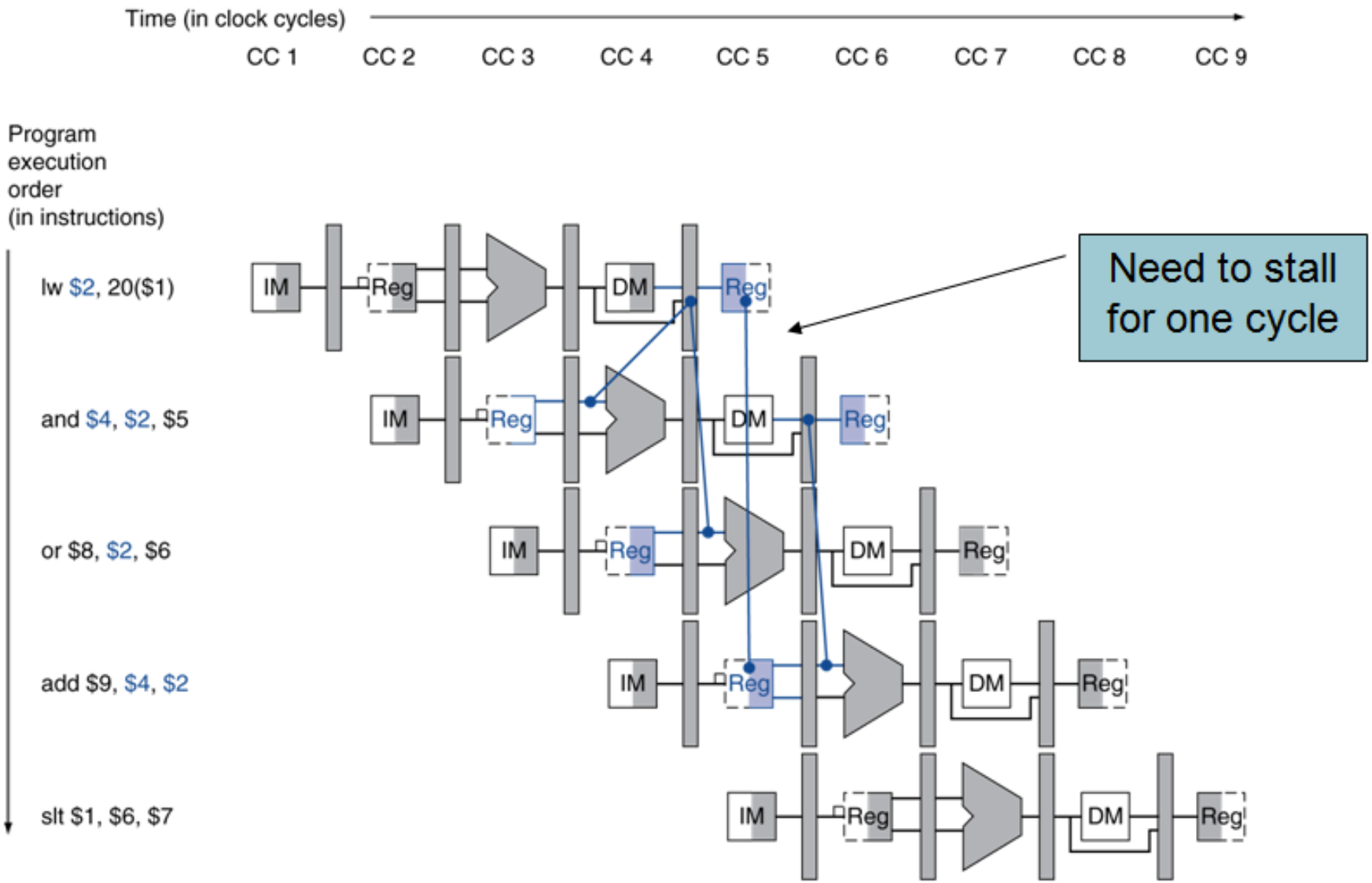
MEM hazard

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
  and (MEM/WB.RegisterRd = ID/EX.RegisterRs))
  and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
  and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
  $\Rightarrow$ ForwardA = [01]
- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
  and (MEM/WB.RegisterRd = ID/EX.RegisterRt))
  and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
  and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
  $\Rightarrow$ ForwardB = [01]

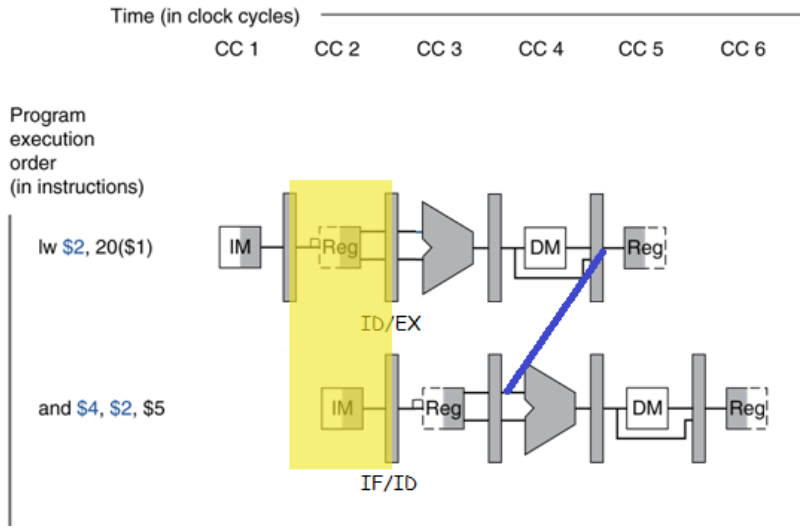
Datapath with Forwarding

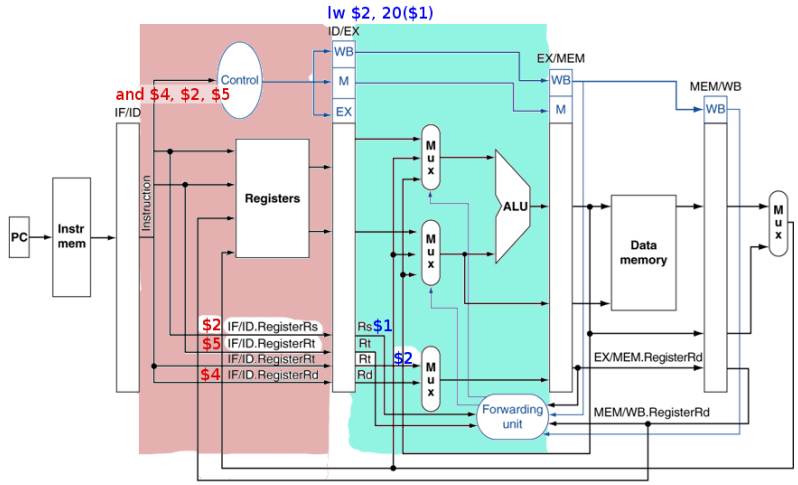


Load-Use Data Hazard



Load-Use Hazard Detection

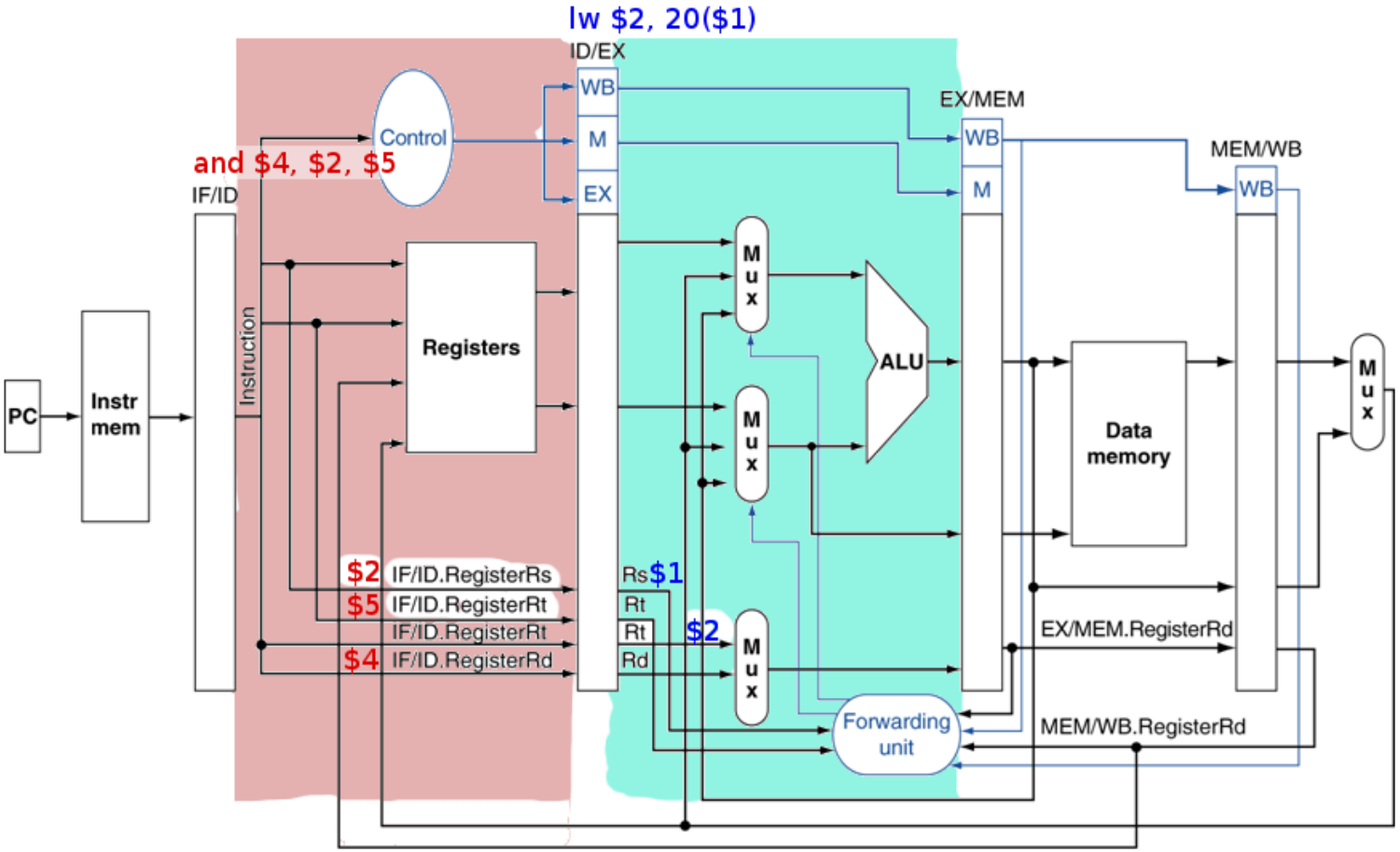




- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
  - IF/ID.RegisterRs, IF/ID.RegisterRt
- Load-use hazard when

  ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs) or (ID/EX.RegisterRt = IF/ID.RegisterRt))

Load-Use Hazard Detection

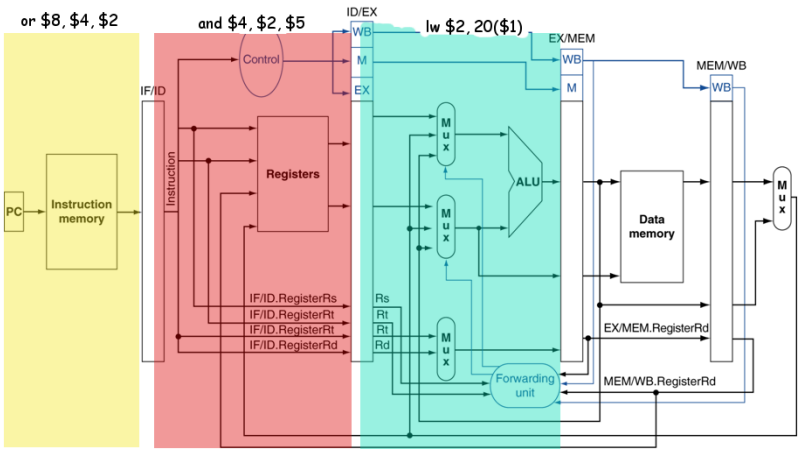


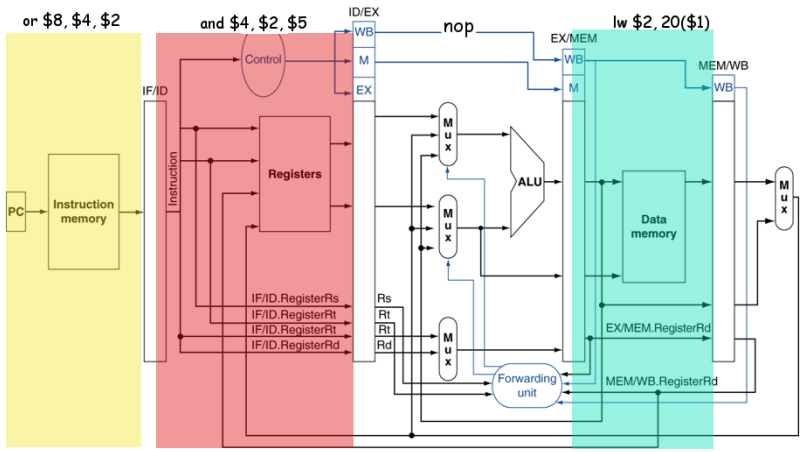
Load-use hazard: ID/EX.MemRead and ((ID/EX.RegisterRt = IF/ID.RegisterRs) or (ID/EX.RegisterRt = IF/ID.RegisterRt)) $\Rightarrow$ Stall and insert bubble

How to Stall the Pipeline

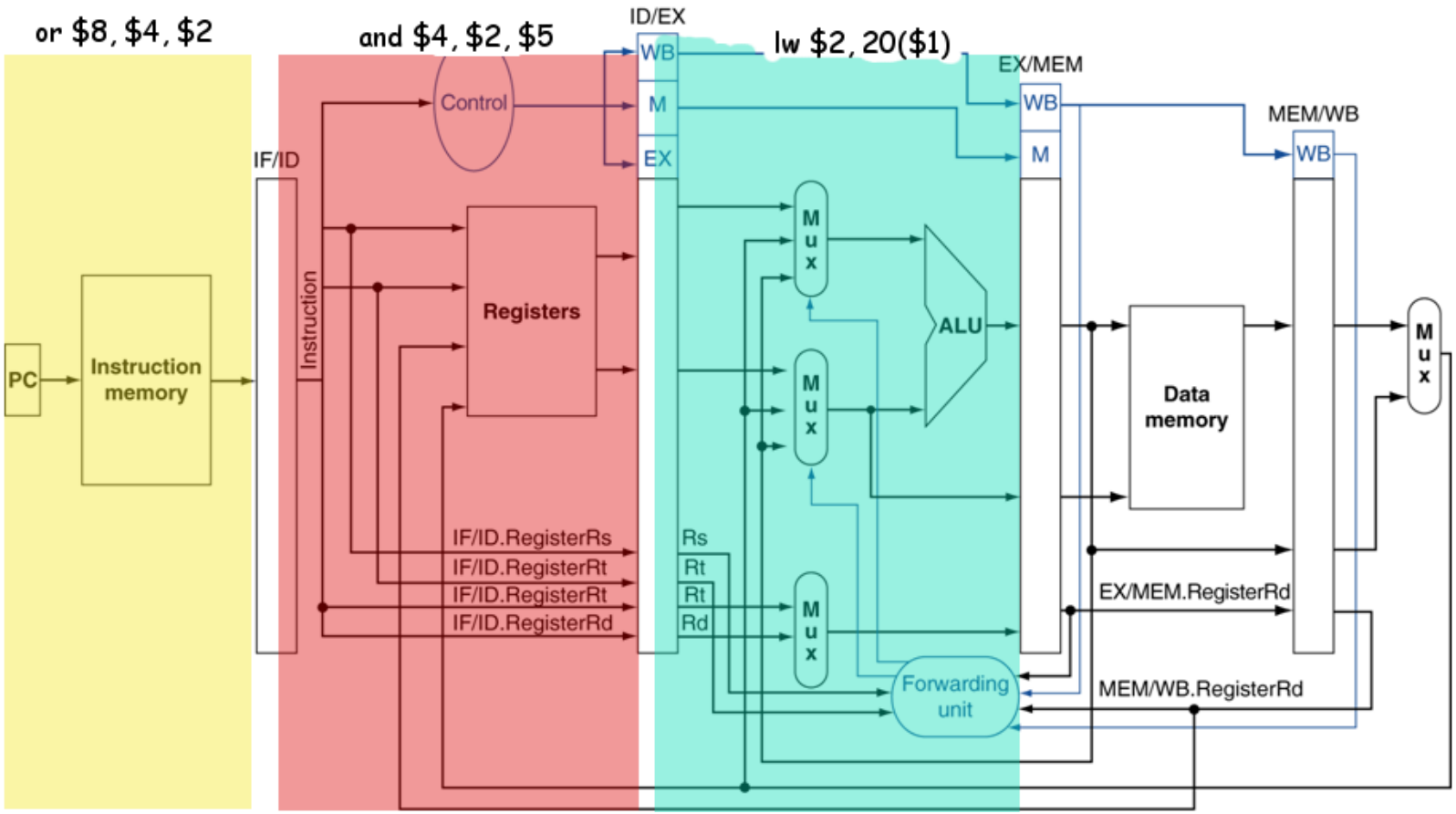
- Force control values in ID/EX register to 0
  - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
  - Using instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for lw

    $\Rightarrow$ Can subsequently forward to EX stage





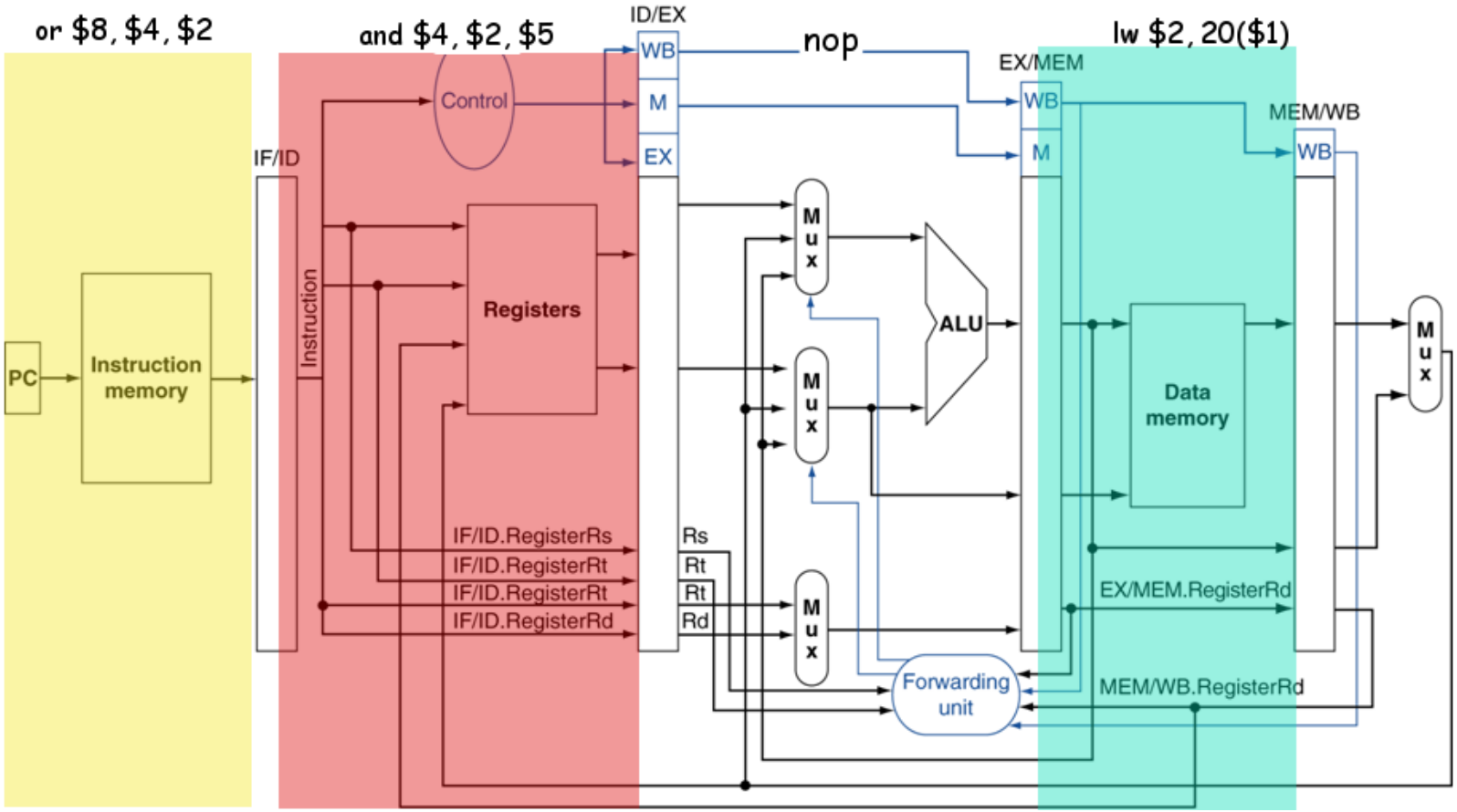
Load-Use Hazard is Detected



ID/EX.MemRead, ID/EX.RegisterRt = 2, IF/ID.RegisterRs = 2

$\Rightarrow$ Load-Use hazard

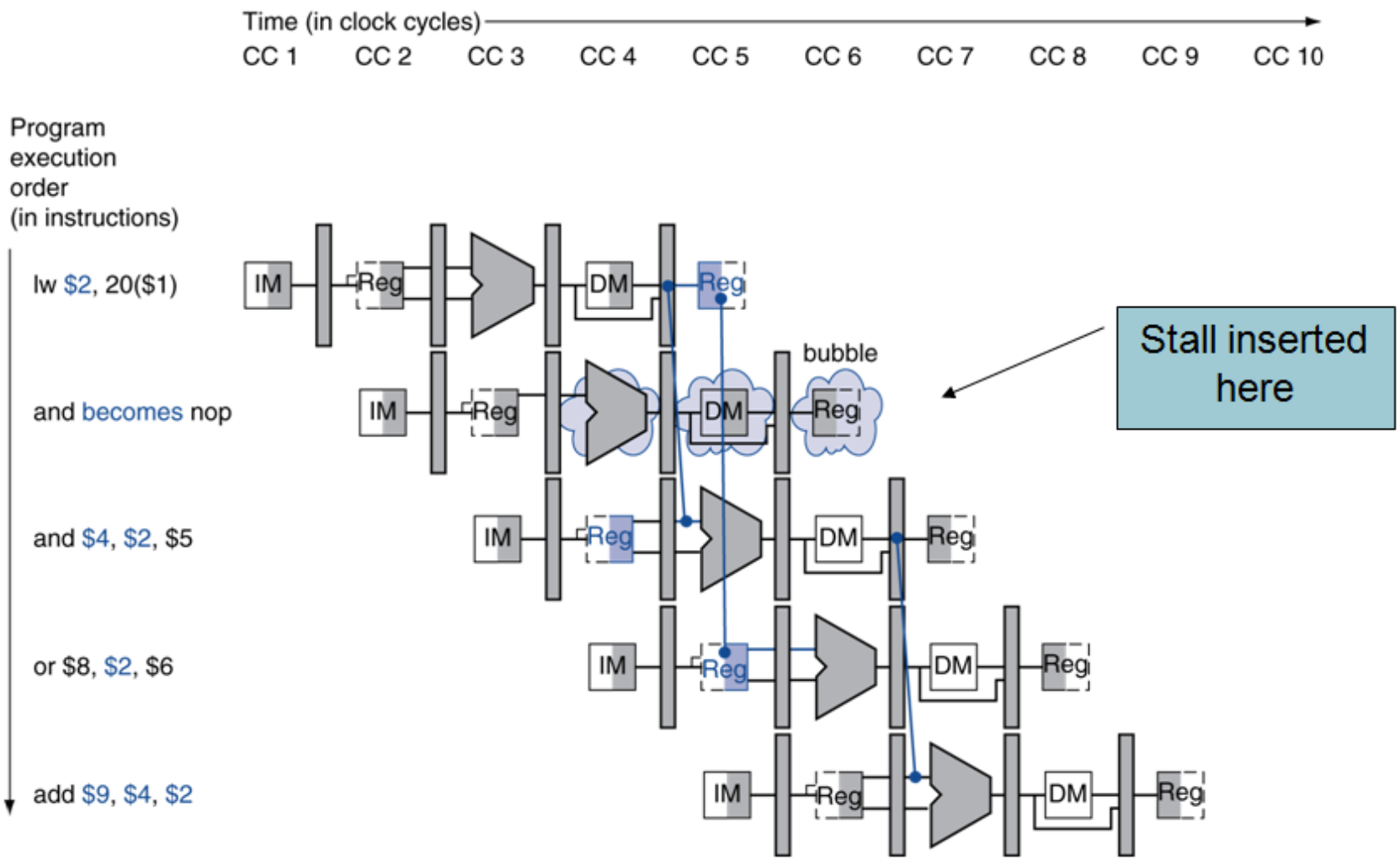
Stall the Pipeline



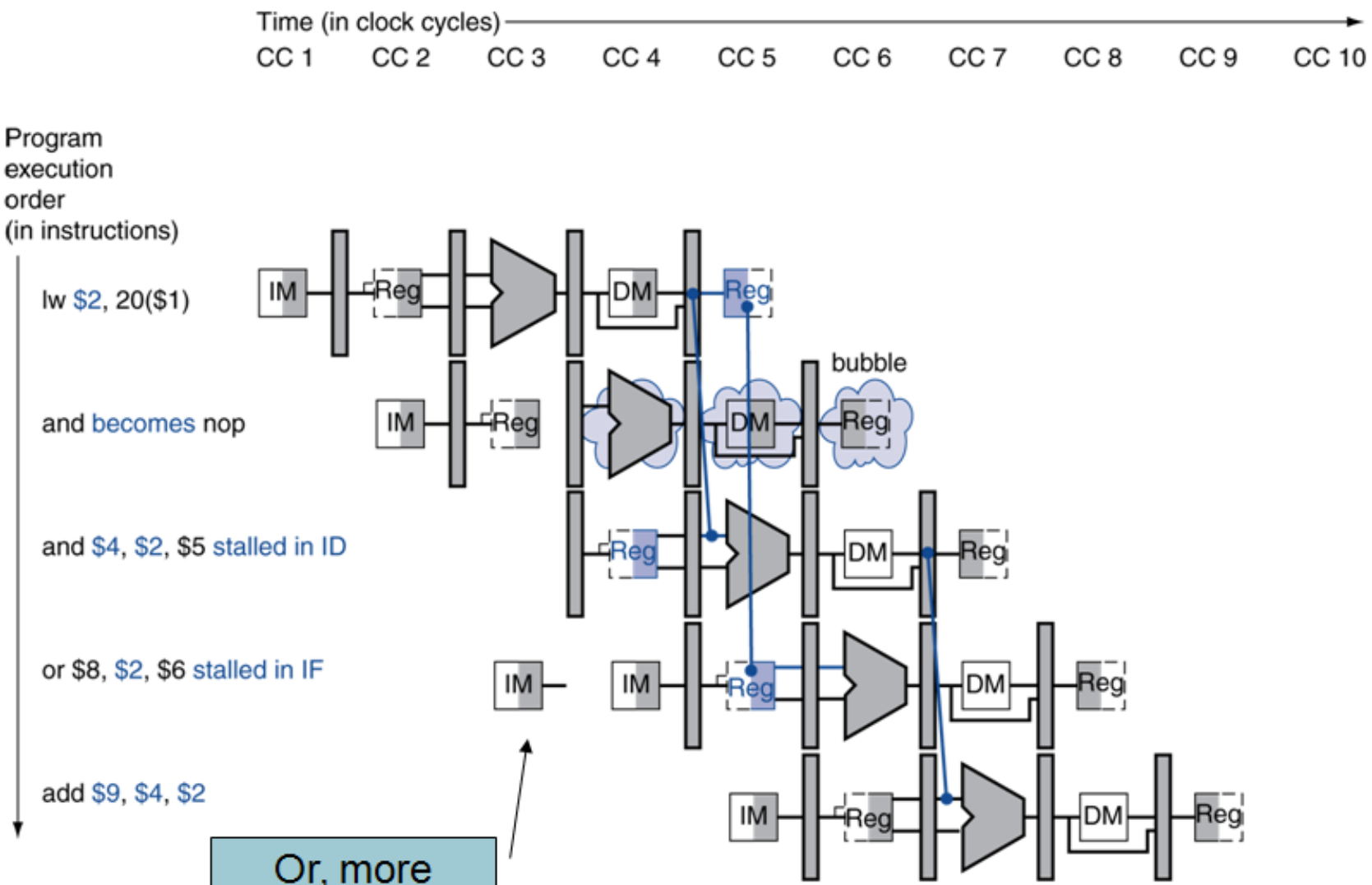
Reset ID/EX to nop

Prevent update of PC and IF/ID

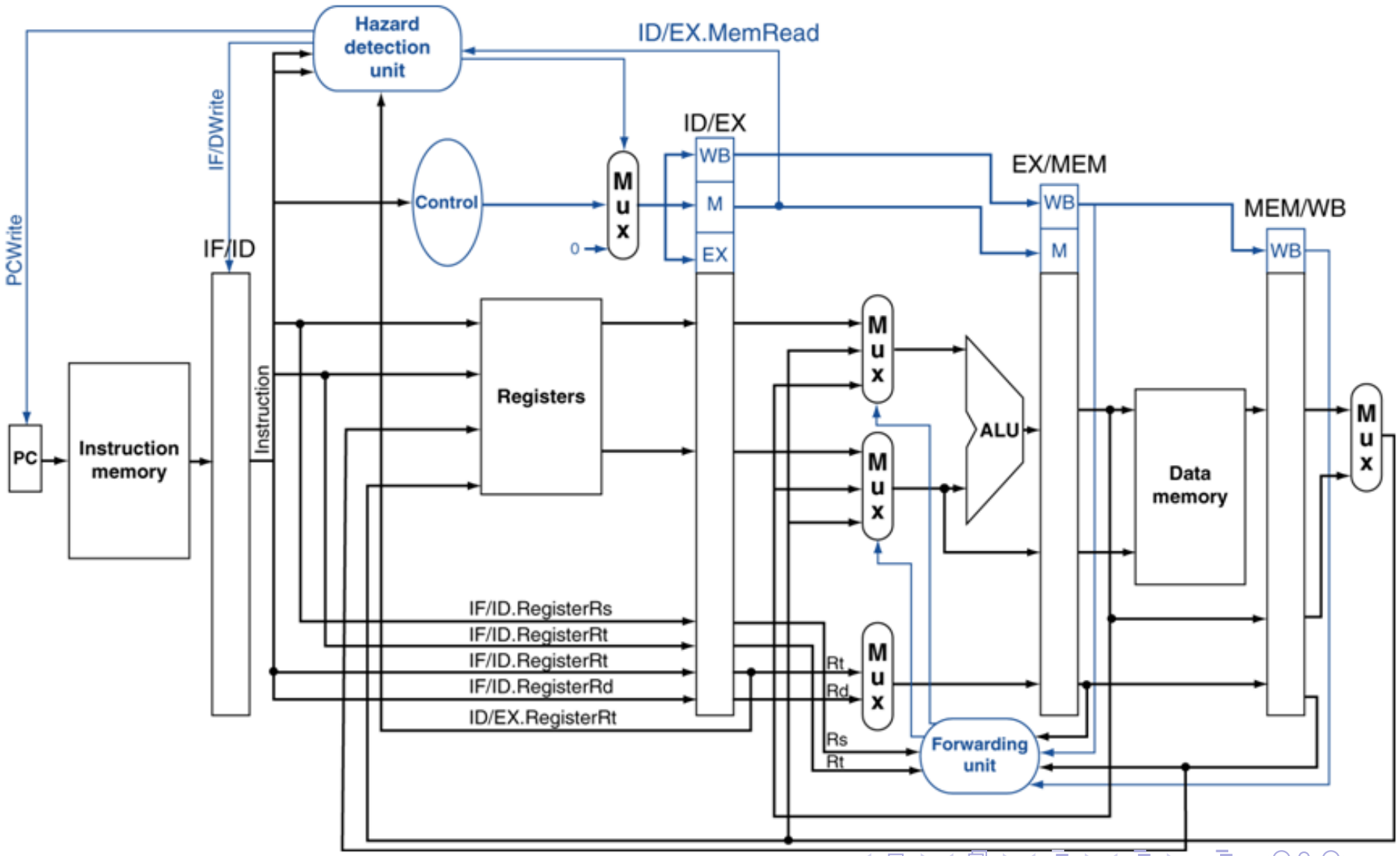
Stall/Bubble in the Pipeline



Stall/Bubble in the Pipeline



Datapath with Hazard Detection

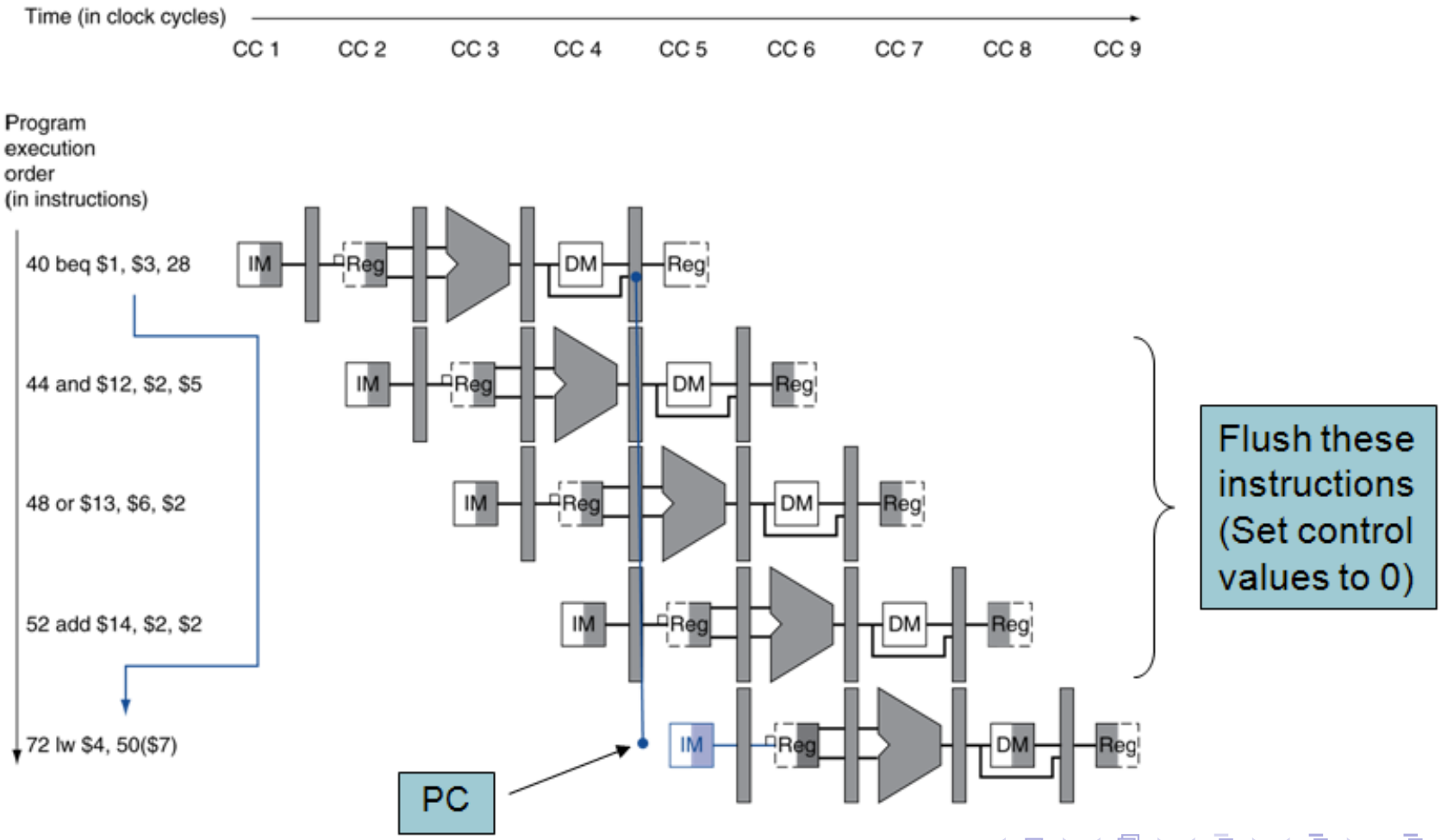


Stalls and Performance

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

Branch Hazards

If branch outcome determined in MEM



Reducing Branch Delay

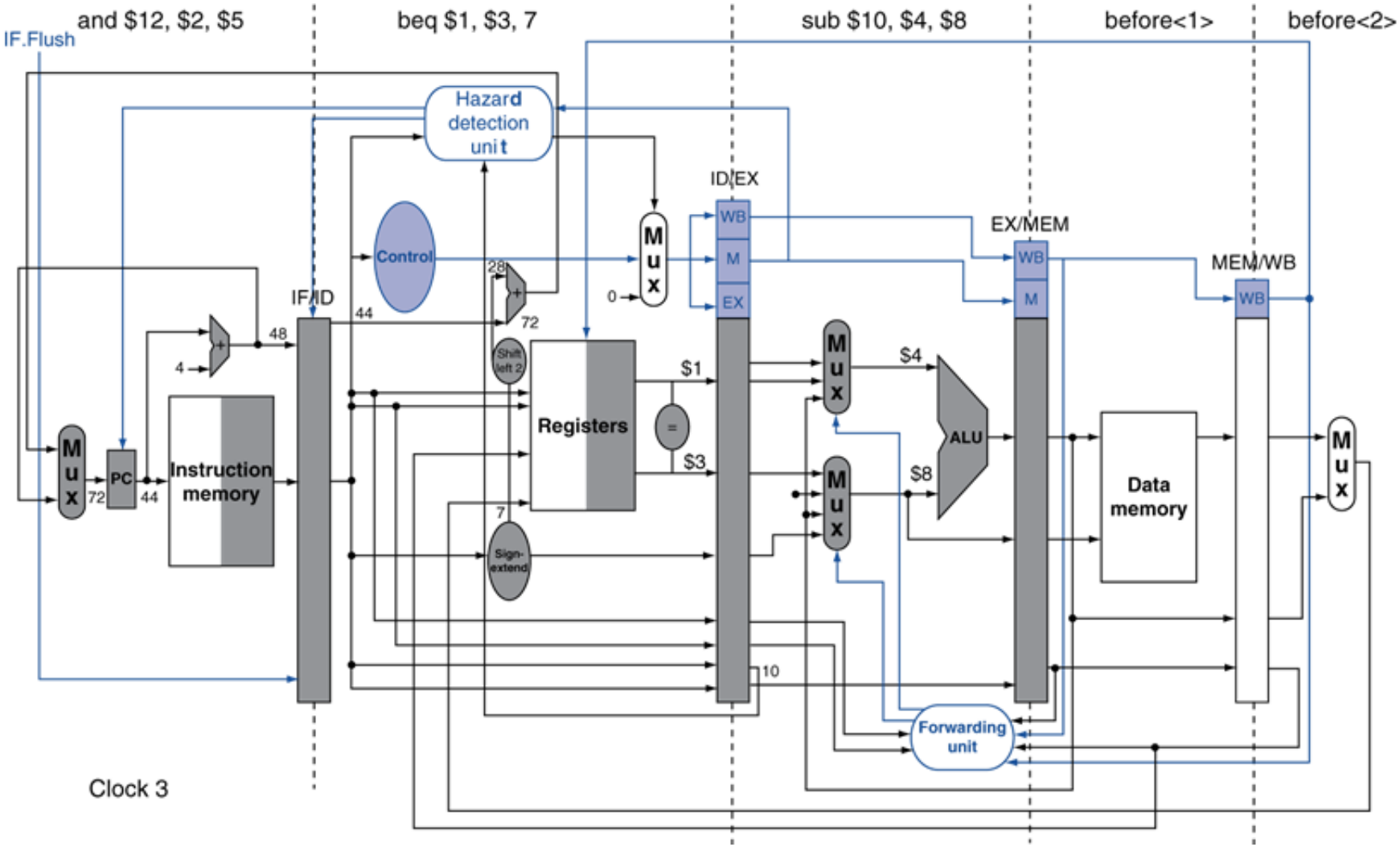
Move hardware to determine outcome to ID stage

- Target address adder
- Register comparator

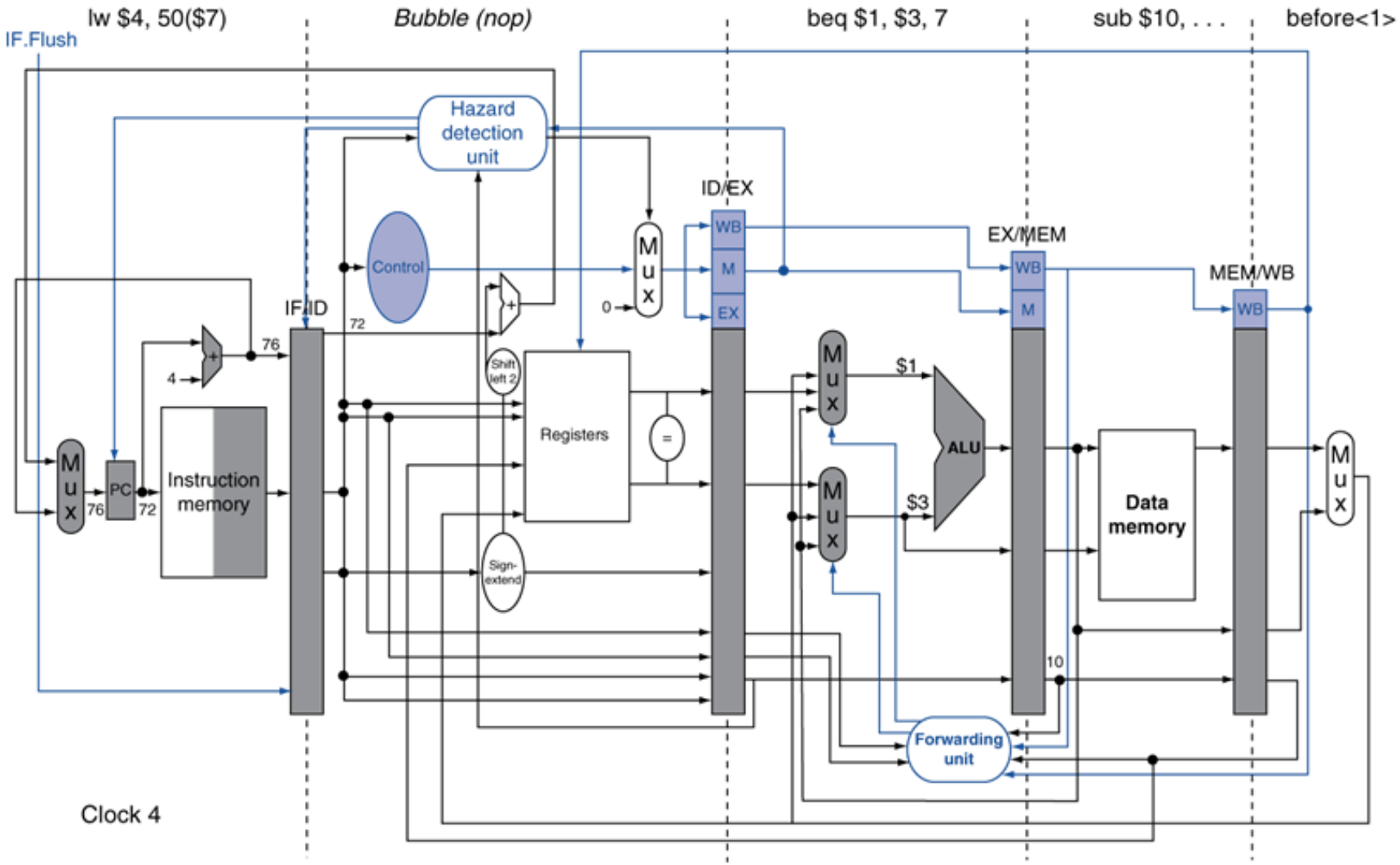
Example: branch taken

```
36:  sub  $10, $4, $8
40:  beq  $1,  $3, 7
44:  and  $12, $2, $5
48:  or   $13, $2, $6
52:  add  $14, $4, $2
56:  slt  $15, $6, $7
     ...
72:  lw   $4, 50($7)
```

Example: Branch Taken (1)



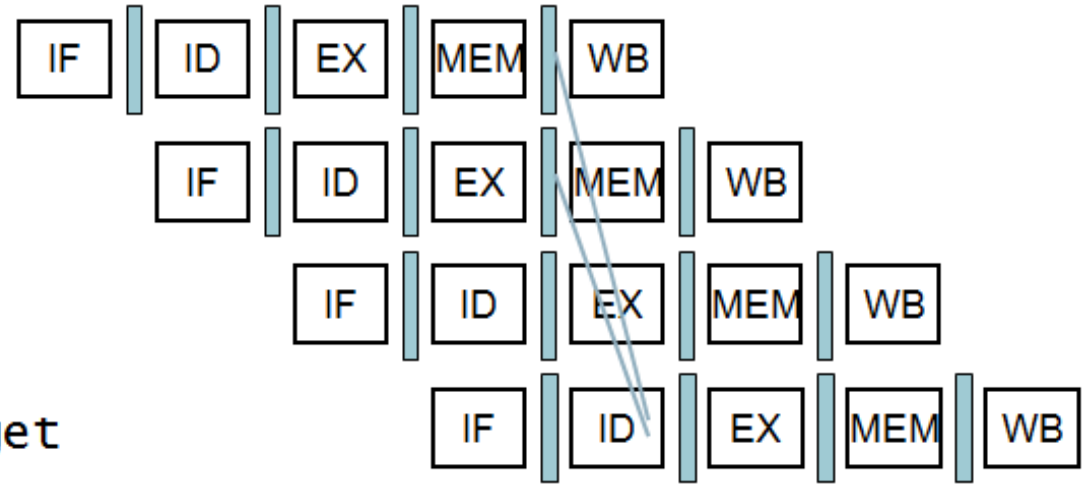
Example: Branch Taken (2)



Data Hazards for Branches

- If a comparison register is a destination of 2nd or 3rd preceding ALU instruction

```
add $1, $2, $3
add $4, $5, $6
...
beq $1, $4, target
```

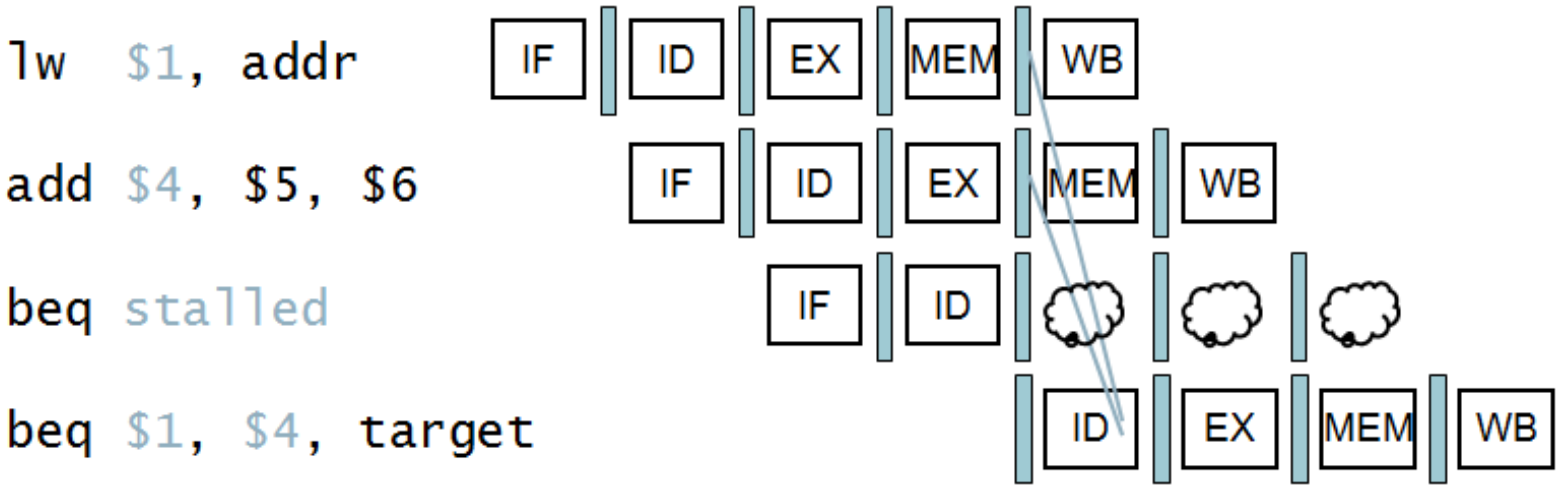


- Can resolve using forwarding

Data Hazards for Branches

• If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction

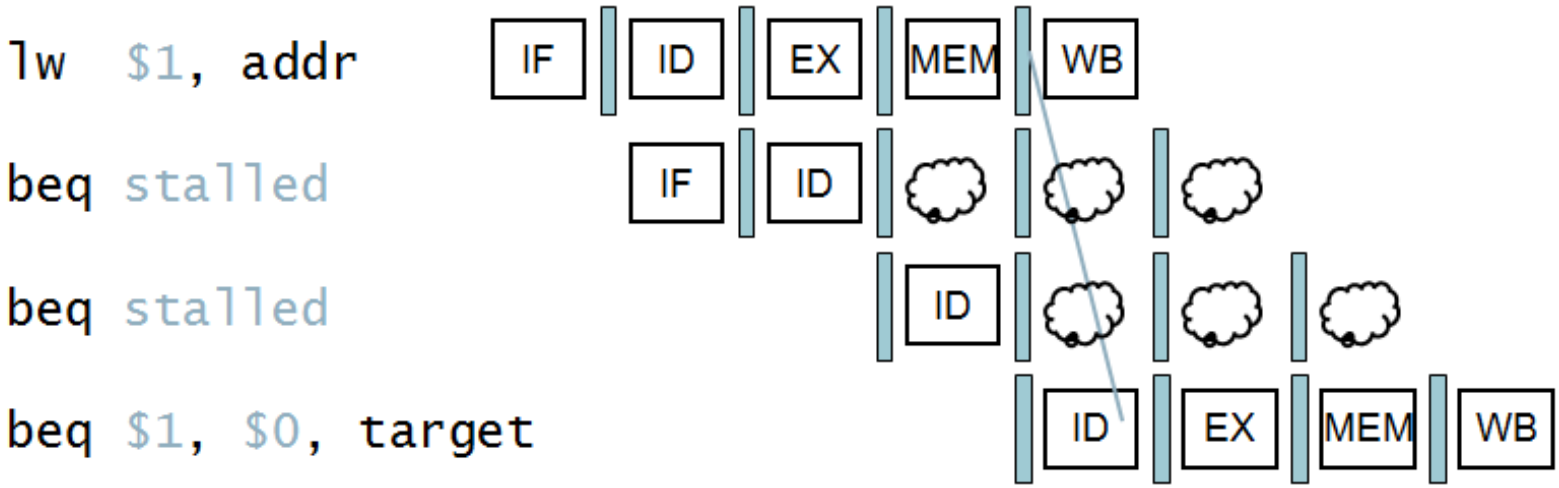
⇒ Need 1 stall cycle



Data Hazards for Branches

• If a comparison register is a destination of immediately preceding load instruction

⇒ Need 2 stall cycles



Closing note

- อ่าน P&H5e, บทที่ 4.
- อ่าน บทที่ 5 เอกสารประกอบรายวิชา 188 322 สถาปัตยกรรมคอมพิวเตอร์, COA notes (April 25, 2014).

